

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE



ROBOTIQUE, 2025-26

Master thesis

DASH-01 :
RL Framework to Develop a Bipedal
Humanoid Robot Aimed at Breaking the
Speed Record

REHASSIST
13.04.2025 - 04.09.2025

Author:

Nathann MORAND
Robotics M.Sc.

Supervisors:

Dr. Mohamed BOURI

Abstract

This thesis documents the development of DASH-01, a custom parallel-linkage biped built to investigate how fast a small, low-cost legged robot can run, and — more centrally — what it actually takes to apply modern reinforcement-learning locomotion methods to a robot that exists, rather than to a simulator. The work covers the full chain: mechanical modelling of a closed-kinematic-loop leg that URDF cannot express, simulation in MuJoCo with explicit loop closure, a measurement campaign that replaced every assumed plant parameter with a measured one, three successive action-space designs for the learned controller (direct torque, per-step position targets, and a policy-tuned central pattern generator with hard-wired reflexes), a base-DOF-release curriculum, and deployment infrastructure on the robot itself — a 200 Hz control daemon on a non-realtime Raspberry Pi, a web-based bring-up instrument, and an always-on flight recorder.

The headline simulation result is a policy that runs at 2.6 m/s on the fully free plant without falling. The headline measurement result is less comfortable: the real robot is 18 % heavier, ~ 15 % weaker, and its actuator position loop is more than an order of magnitude slower than the model every policy was trained against, which moves the robot’s credible ceiling to roughly 2.7 m/s burst and 1.3 m/s sustained — a walking-and-jogging demonstrator, not a record platform. The thesis argues, with evidence, that this gap is the normal product of simulation-first development, and that the countermeasures are methodological: measure early, treat every training aid and reward term as an exploit target, evaluate on fixed tasks with paired seeds, and instrument the hardware so that failures are reconstructible. A hand-designed classical controller, built late in the project as a baseline, beats the learned policy on the constrained plant and fails at exactly the same wall on the free plant — demonstrating that the binding constraint is the machine, not the learning algorithm.

Table of Contents

Abstract	2
1 Introduction	5
1.1 Problem statement	5
1.2 Objectives of the Project	5
1.3 Methodology Overview	5
1.4 Contributions	5
2 Background and State of the Art	7
2.1 Dynamic biped locomotion	7
2.2 Control approaches	7
2.3 RL for locomotion	7
2.4 Existing robot platforms	7
3 Development Methodology	8
3.1 Overall methodology	8
3.2 Hardware selection	8
3.2.1 Motor	8
3.2.2 Battery pack	9
3.2.3 Compute	10
3.2.4 Sensors	10
3.3 Robot modelling	11
3.4 Simulation	13
3.4.1 URDF cannot describe this robot	13
3.4.2 Compute backend	13
3.4.3 How do you know a simulation result is real?	13
3.5 Plant identification: replacing assumptions with measurements	14
3.5.1 Mass	14
3.5.2 Peak torque is what the gearbox delivers, not what the datasheet quotes	15
3.5.3 The drive's position loop is 0.8Hz	15
3.5.4 The ankle spring	16
3.5.5 System identification: including what it could not resolve	16
3.5.6 Loop timing and CAN	17
3.5.7 The CAN protocol, as measured	18
3.6 Achievable gait: actuator and mass-distribution limits	18
3.6.1 Method	18
3.6.2 Results	19
3.6.3 Limiting factors	19
3.6.4 Sensitivity to mass distribution	20
3.6.5 Ankle stiffness	20
3.6.6 The corrected ceiling	20
3.6.7 Limitations	21
3.7 Toy problem: one joint, end to end	21
3.8 RL policy design and validation	21
3.8.1 Attempt 1: direct joint torque	22
3.8.2 Attempt 2: joint position targets (PD)	23
3.8.3 Attempt 3: a central pattern generator tuned by the policy	24
3.8.4 Reflexes: what is learned and what is hard-wired	26

3.8.5	The pitch-balance wall, and how it was broken	27
3.8.6	The cadence problem and a cautionary bug	29
3.8.7	Results	30
3.8.8	Which of these results survive the plant correction	31
3.8.9	Validation status and known gaps	32
3.8.10	A second generator: explicit-oscillator CPG versus the Fourier series . . .	33
3.8.11	The ankle, revisited as a build decision	34
3.8.12	The drivable-demonstrator lineage: closing on the measured plant	34
3.8.13	A scripted baseline: how far does no learning get?	36
3.8.14	Postscript: the rebuilt robot, and a cap that was never there	37
3.9	Negative results	38
3.10	Deployment	39
3.10.1	Linux without a real-time kernel	39
3.10.2	CAN at runtime: the bus with nothing on it	40
3.10.3	The motor controller as deployed	41
3.10.4	The safety envelope as an instrument	41
3.10.5	Hardware validation: the flight recorder and the pre-move guard	42
3.11	Hardware experiments	44
3.11.1	Static end-effector force map	45
4	Results	46
4.1	Hardware	46
4.2	RL experiments	46
5	Discussion	47
5.1	Successes	47
5.2	Failures	47
5.3	Lessons learned	47
6	Conclusion and Future Work	49
6.1	Summary of Findings	49
6.2	Challenges and Limitations	49
6.3	Suggestions for Future Research	49
7	Appendix	51
7.1	logbook	51
7.2	URDF from CAD:step by step	51
	References	52

1 Introduction

1.1 Problem statement

Reinforcement learning has produced striking legged-locomotion results on a small set of well-instrumented research platforms. Reproducing those results on a new, custom, imperfectly known robot is a different problem, and it is the problem most robot builders actually face: the plant parameters are uncertain, the actuators have undocumented internal control loops, the sensors are commodity parts, and the compute is a single-board computer. The gap between “RL walks in simulation” and “RL walks on this machine” is dominated not by learning algorithms but by modelling fidelity, actuation bandwidth, calibration, and instrumentation.

DASH-01 is a two-legged, six-motor robot with a parallel-linkage knee and a passive ankle, designed around the ambition of high running speed. This thesis takes it from a partially finished mechanical design to a measured, simulated, learning-trained and hardware-instrumented platform, and reports — deliberately including the negative results — what each stage cost and what it revealed.

1.2 Objectives of the Project

1. Model the robot faithfully enough to train against: kinematics (including the closed 4-bar loop), masses, actuators, sensors.
2. Establish the robot’s physical performance envelope independently of any controller, so learned results can be judged against a ceiling rather than against hope.
3. Train locomotion policies with reinforcement learning, iterating on the action-space design until one converges reliably, and quantify what remains between those policies and hardware deployment.
4. Build the deployment stack — control loop, calibration, safety, telemetry — on the real robot, and validate as much of the chain as the hardware state allows.
5. Record the methodology honestly enough to be reproducible, including the failures.

1.3 Methodology Overview

The project is a hybrid start: the mechanical design existed in part, and the work here begins at modelling and hardware selection for the non-mechanical subsystems. The development order was: CAD re-assembly and export to a simulable model; validation of the model against the physical robot (workspace, statics, then full dynamic identification); a first-principles capability analysis on the validated model; a toy problem (a single-joint inverted pendulum) to exercise the complete train–export–deploy chain at minimum stakes; three generations of RL policy design trained on an HPC cluster; and finally the deployment stack on the robot, driven by a measurement campaign that fed corrections back into the simulation. Section 3.1 details the sequence; the single most consequential methodological finding is that *every* assumed plant parameter that was later measured moved in the pessimistic direction (Table 3.4).

1.4 Contributions

- A complete CAD-to-MJCF modelling pipeline for a closed-loop-linkage biped, including the loop-closure treatment URDF cannot express, and validation tooling (analytic Jacobians

checked against finite differences, loop closure checked against the measured backdriven workspace at 100 % of 2789 poses).

- A measured plant: weighed segment masses, identified torque constants via a known-mass method that breaks the K_t /gravity degeneracy, the drive’s internal position-loop Bode (0.8 Hz + 25 ms), IMU noise with provenance labels, loop timing, and CAN protocol — together with an explicit statement of what the identification could *not* resolve.
- A first-principles running-speed capability analysis validated against MuJoCo, with a mass-location sensitivity result (distal mass $\sim 9\times$ costlier than torso mass) that later measurements repeatedly confirmed as the binding design axis.
- A policy architecture — learned Fourier gait generator re-parameterised per step, mirrored across legs, plus hard-wired and learned reflexes and a per-step residual channel — and a base-DOF-release curriculum that trains it, with an A/B against an explicit-oscillator CPG generator on identical plants and rewards.
- A catalogue of failure modes with measured diagnoses: reward exploits (skating, floor penetration, shuttling, suicide-by-penalty), curriculum deadlocks, evaluation-restore bugs, a warm-start seed bug that silently duplicated every seed replicate, and the pitch-balance wall traced to plant statics rather than learning.
- A scripted classical baseline (capture-point + clocked gait, 21 gains, no learning) that outperforms RL on the constrained plant and fails identically on the free plant — evidence that the wall is physical.
- Deployment infrastructure on the robot: a 200 Hz daemon on stock Linux, a web bring-up instrument through which most of the measurement campaign was performed, and a three-tier flight recorder with a pre-move safety guard, built in response to a hardware failure and validated by reproducing that failure.

2 Background and State of the Art

2.1 Dynamic biped locomotion

Running differs from walking in that the centre of mass follows a spring-like, not vaulting, trajectory, with flight phases and ground contacts of tens of milliseconds. The reduced models that organise the field — the inverted pendulum for walking, the spring-loaded inverted pendulum (SLIP) for running, and capture-point/instantaneous-capture-point analysis for push recovery — appear throughout this thesis in load-bearing roles: the capture point underlies both the hand-wired pitch reflex (Section 3.8.4) and the scripted baseline (Section 3.8.13), and the free-topple time of the inverted pendulum turns out to *be* the m3 failure signature (Section 3.8.13). Stabilisation is hard because the dynamics are underactuated during flight and contact, the useful control inputs (foot placement, CoP modulation) are discrete or saturating, and the timescales are short relative to actuator bandwidth — the last of which this project measured the hard way (Section 3.5.3).

2.2 Control approaches

Classical stacks decompose the problem: a gait/footstep planner, a whole-body or centroidal controller, and joint-level tracking. They are transparent and analysable but demand accurate models and high-bandwidth torque control. Learned controllers collapse the stack into a policy trained in simulation, trading analysability for robustness to the messiness of contact — at the price of a sim-to-real gap that must be closed with modelling fidelity and domain randomisation. This thesis ends up using both: RL for the gait, and classical structure (reflexes, a clocked generator, capture-point priors) exactly where feedback must not be blind (Section 3.8.4). The scripted baseline of Section 3.8.13 is the classical counterfactual, built on the same plant to answer “was the learning ever the bottleneck?”

2.3 RL for locomotion

The relevant lineage for this work is the family of *periodically parameterised* action spaces: Policies-Modulating-Trajectory-Generators (PMTG) [3], oscillator-based CPG-RL [1], and the clock/phase-reward methods of Siekmann et al. [4], which carried Cassie to a 100 m Guinness record (24.73 s, ≈ 4.1 m/s average) [2]. A literature review conducted mid-project (Section 3.8.3) returned one unambiguous constraint that reshaped the architecture here: **no published system achieves running with per-cycle-only gait-parameter updates and no intra-cycle feedback channel** — every successful system keeps a per-step path from observation to action. The same review supplied the phase-gated contact rewards and efficiency-term designs adopted in Section 3.8.3. On the sim-to-real side, the standard toolkit — domain randomisation with curricula, observation history in place of privileged state, explicit sensor models — is exercised in Section 3.8.12, where an ablation shows which of its components actually carried the cost.

2.4 Existing robot platforms

3 Development Methodology

3.1 Overall methodology

The project inherited a partially designed mechanical platform and proceeded in the following order, each stage gating the next:

1. **Modelling and hardware selection.** Re-assembly of the CAD, export to a simulable model (Section 3.3), and selection of the non-mechanical hardware: compute, sensing, battery (Section 3.2).
2. **Model validation.** Workspace calibration on the physical robot, static validation, and — much later, with better instrumentation — full dynamic identification (Section 3.5). The lesson of the project in one sentence: this stage was done too late and too lightly at first, and every subsequent correction moved the plant in the pessimistic direction.
3. **Capability analysis.** A first-principles gait-feasibility sweep on the validated model (Section 3.6), so that learned results have a ceiling to be judged against.
4. **Toy problem.** A single-motor inverted pendulum carrying the complete train–export–deploy loop (Section 3.7) before committing the 12-DOF robot to the same pipeline.
5. **RL policy development.** Three action-space generations, a reflex architecture, a base-DOF-release curriculum, and a generator A/B — trained on the EPFL Izar cluster (Section 3.8).
6. **Deployment.** The 200 Hz control stack, calibration and safety machinery, and the flight recorder on the robot itself (Section 3.10), plus the measurement campaign that fed the corrections of Section 3.5 back into every earlier stage.

Two cross-cutting disciplines emerged as more important than any single stage: *adversarial review of anything that produces a number* (rewards, simulators, estimators and evaluators all produced confident wrong answers before being caught — Sections 3.4.3 and 3.5.5), and *measurement before assumption* (three project-level decisions — the torque-control abandonment, the 50 Hz control rate, the ankle actuation question — were each made on assumed numbers and each reversed or reframed by later measurement).

3.2 Hardware selection

3.2.1 Motor

Each leg carries three actuators, all CubeMars quasi-direct-drive units with integrated drivers speaking a servo-mode CAN protocol:

	Hip roll (abduction)	Cam and thigh (sagittal)
Part	CubeMars AK60-39	CubeMars AKE90-8
Reduction	39:1	8:1
Peak torque (datasheet)	72 N·m	170 N·m
Continuous torque	—	55 N·m
No-load speed	10.3 rad/s	22 rad/s
Motor K_t	(unpublished)	0.272 N·m/A
Reflected armature	0.046 kg·m ²	0.0216 kg·m ²
Mass	0.75 kg	1.40 kg

Table 3.1: The six actuators (two AK60-39, four AKE90-8), datasheet values. Section 3.5 measures the effective joint K_t at 82–89 % of these figures ($\approx 85\%$ gearbox efficiency), so the deployed peak torques are 61.2 and 144.5 N·m respectively.

The no-load speeds follow from the 48 V bus: the AK60-39 is a KV80 motor, so $80 \text{ rpm/V} \times 48 \text{ V}/39 = 10.3 \text{ rad/s}$ at the joint; the AKE90-8’s 22 rad/s implies $KV \approx 35$. The sagittal pair dominates the robot’s capability envelope: the capability analysis of Section 3.6 finds 87 % of feasible gaits limited at the cam actuator’s peak-power corner ($P \approx \tau_0 \omega_0/4 \approx 935 \text{ W}$), which makes *no-load speed*, not peak torque, the axis a successor should buy. The low reduction ratios keep reflected inertia low enough that the legs can be backdriven by hand — the property the teach-and-replay gait recording of Section 3.11 depends on — but the integrated drivers turned out to carry the project’s largest hidden constraint: an internal, proportional-only position loop with a 0.8 Hz bandwidth that no runtime command can retune (Section 3.5.3).

One deliberate modelling omission is documented rather than papered over: the simulation applies independent torque and speed caps but *no coupled torque–speed curve*, because the phase resistance R has never been measured on this robot. The bound $R \leq K_t V/\tau_{\text{peak}}$ (0.72Ω for the AKE90-8, 3.65Ω for the AK60-39) places both motors on the flat branch of the curve at every *observed* joint speed, so a constant force range is currently correct — but the omission is optimistic exactly in the fast-running regime the project targets, and is flagged accordingly in Section 3.6.

3.2.2 Battery pack

The motor selection fixes the electrical requirement: six drives on a nominal 48 V bus, with short, violent current peaks — a sprint robot’s duty cycle, not a marathon robot’s. (Usain Bolt needs under ten seconds for 100 m; this robot would be delighted with thirty.) The pack is therefore optimised for *power* density rather than energy density.

Cell chemistry was reviewed on that criterion. LTO and high-rate LiPo chemistries are nominally more power-dense, but no reputable supplier could be found that ships bare cells of either — commercial LTO cells surfaced only on AliExpress and were judged not worth the provenance risk, and packaged LiPo modules could not be sourced at 48 V. The practical supplier space (Nkon, the supplier used in past lab projects) reduced the choice to high-rate 21700 NMC cells, of which the **EVE INR21700-50PL** (5.0 Ah, 125 A rated discharge) was the most power-dense available; cost was a secondary criterion pointing the same way.

The pack is **13S2P** (48.1 V nominal, 10 Ah, $\approx 480 \text{ Wh}$), giving a cell-limited 250 A. The BMS is rated 200 A continuous — thermally limited — with the peak capped at 250 A in software.

One consequence of the *drive* selection only became visible once the CAN traffic was sniffed (Section 3.5.7): the motor controllers publish position, speed, current, temperature and an error flag, but **no bus voltage**. Pack sag under a sprint is therefore not observable from the drives at

all, and would need external sensing on the Pi or the BMS. Until that exists, voltage droop is modelled in simulation as a randomised within-episode torque derating (Section 3.8.12) rather than as a measured quantity.

3.2.3 Compute

Most of the lab exoskeleton run on beaglebone black While theses support CAN natively, and run linux, the SSH connection was VERY slow and thus was rejected. The main selling point was the real time coprocessor (understand a beefy MCU that run alongside the main CPU for time critical application) but this require custom code and the performance benefit are still to demonstrate.

A Raspberry Pi that was laying around was used instead as the author had much more experience with theses. It did require to buy a CAN hat over SPI. another hat with an accelerometer was bought (Waveshare Sense HAT (B)) for the ease of integration.

mention here or later the need to check the mapping when stacking the two hats.

The board is a **Raspberry Pi 3 Model B Rev 1.2** ($4\times$ Cortex-A53, aarch64, kernel 6.12), running a stock non-realtime Linux. CAN is provided by an `mcp251xfd` controller over SPI, configured as **two independent 1 Mbit buses**: `can0` serves the right leg and `can1` the left, three motors each (CAN ids 104 abduction, 105 cam, 106 thigh). The arbitration id encodes both node and command as `id | (command << 8)`.

The Pi also hosts a WiFi access point and a web interface used to drive the robot during bring-up, which removes the need for a tethered console. That web interface is not a convenience only: it is the instrument through which most of the measurements in Section 3.5 were taken.

The loop-rate and jitter question this section originally left open is settled by measurement in Section 3.5.6: the Pi sustains a 200 Hz closed control loop at half its time budget, and 200 Hz is the correct deployment rate.

3.2.4 Sensors

The only proprioceptive sensors beyond the motor encoders are the base IMU and the motor current estimates. Contact sensors were considered, since the literature suggests that impact timing and direction improve RL convergence, but were never fitted.

The IMU is an **ICM-20948** on a Waveshare Sense HAT (B), bolted underneath the base and read over I2C. It is configured at $\pm 4g$ and ± 1000 dps with the internal low-pass at a 50 Hz bandwidth, sampled at 200 Hz. Its measured properties are summarised below; the full characterisation, with the provenance of every figure, is in the project repository.

	Accelerometer	Gyroscope	Source
Resolution (1 LSB)	0.1221 mg	0.0305 dps	measured
Noise (RMS at rest)	1.8 mg	0.08 dps	measured at rest
Noise density	$218 \mu g/\sqrt{Hz}$	$0.0096 \text{ dps}/\sqrt{Hz}$	at or better than datasheet
Turn-on bias	absorbed by mount cal.	$[1.03, 0.11, -0.53]$ dps	measured, repeatable
In-run bias stability	—	0.010–0.012 dps	measured

Table 3.2: Measured IMU noise characteristics. Quantisation sits roughly $15\times$ below the noise floor on both channels, so widening the ranges to survive footfall impacts costs nothing in effective resolution.

Two properties matter more for control than the noise figures do.

Latency. The path from a physical event to a published value is 10–20 ms: DLPF group delay (5–10 ms, datasheet, not verified on this part), output-rate quantisation (0–4.3 ms), the I2C read (1.7 ms) and the poll period (0–5 ms). At a 5 ms control period that is two to four whole control steps, which is a larger effect than the sensor noise and is modelled as such in simulation.

The attitude filter dominates, not the sensor. A 6-axis Madgwick filter with $\beta = 0.06$ rad/s runs at the poll rate. Its accelerometer correction is rate-limited to roughly $3.4^\circ/\text{s}$ regardless of the size of the error, so the published attitude is overwhelmingly gyro-driven and the accelerometer acts as a slow trim. Roll and pitch are gravity-referenced and absolute; yaw is pure gyro integration and drifts, so the gyro zero must be captured on a stationary robot each session. If a policy consumes fused attitude rather than raw rates, the simulation has to model this filter and not merely the sensor.

The magnetometer is deliberately not fused: it sits centimetres from two brushless motors and the battery, and its reading moves when they do.

A methodological trap worth recording. Noise figures taken while the robot swayed on its test rig measure the sway, and nothing in the resulting number says so. That mistake produced a yaw-axis noise figure roughly $8\times$ too high, and a filter sweep in which a *narrower* low-pass appeared to measure *more* noise. Both results looked like plausible sensor data. The benchmark tool now gates every noise figure on a peak-to-peak motion check, and every published figure carries a provenance label distinguishing measured-at-rest, motion-independent, datasheet-unverified and uncharacterised. The uncharacterised list is published rather than guessed at: temperature dependence of the gyro bias, vibration rectification under footfall impact, accelerometer cross-axis sensitivity, and clipping behaviour at ± 4 g during real impacts.

Getting the rate the sensor was configured for. The IMU loop initially reached only 190–194 Hz of a nominal 200. The missing ticks were not the IMU’s: the air, pressure, light and ADC channels were being read inline on the same thread, and those drivers spend nearly all their time asleep waiting for a conversion. The humidity sensor alone sleeps 15 ms, three whole control ticks. Moving the slow chips to a second thread, with a lock serialising only the bus transfers, brought the IMU to 199.5 Hz and reduced the CAN daemon’s late ticks from 11.5 % to 9.8 % as a side effect. The remaining slips are operating-system scheduling.

3.3 Robot modelling

The robot was re-assembled in Autodesk Inventor (chosen for fluency), each limb exported as STEP with its origin centred on its joint, then assembled in Fusion 360 and exported to URDF with a plugin. The path was not smooth, and the detours are worth recording because they are the normal experience of CAD-to-simulation work:

- **Inertia matrices from the mechanical engineer’s SolidWorks model arrived in the wrong orientation**, and after multiple attempts the SolidWorks URDF export never yielded a usable file. The working pipeline validates every CAD-supplied inertia tensor against its reference frame explicitly rather than trusting the export.
- **URDF cannot express this robot at all** (Section 3.4): the parallel knee closes a kinematic loop, so the URDF is at best an open-tree approximation with a floating pushrod, and MJCF with an explicit `<equality><connect>` is the actual model. Both are exported and kept; only the MJCF is simulated.

- **Validation is a tool, not an inspection.** A dedicated tool discovers plausible inertia matrices from the physical robot (moving the legs in the air with the base fixed, inside the calibrated safe workspace) and compares them against the CAD values. The full dynamic identification this grew into is reported in Section 3.5.5; its bottom line for the model is that the CAD centres of mass *survive* scrutiny (a free 18-parameter fit explains at most 11–19% of the observed gravity residual, so the CAD values were kept), while the link inertia *tensors* remain CAD placeholders scaled by measured mass ratios — which is why the domain-randomisation range on inertia stays at $\pm 25\%$ while the mass ranges narrowed.
- **The kinematic model is consistent with the measured robot:** the MJCF closes the 4-bar at 100% of the backdriven-workspace poses recorded on the real machine (1383/1383 left, 1406/1406 right, residual 10^{-14}) — a one-sided test (it would only catch a model band *smaller* than reality), but a reassuring one.
- **The exported model is not left/right symmetric, and symmetrising it was tried and reverted.** An audit found 17 of 30 mirror-compared quantities off — a 1.2° rotation in one hip frame, sub-millimetre CoM offsets — which the near-singular 4-bar amplifies into 6.5° of hip-roll difference in the standing keyframe. A tool that mirrors the model exactly (worst error $1.2^\circ \rightarrow 2 \times 10^{-5}$) was built, and *not applied*: the crooked stance survives symmetrisation (the gravity settle bifurcates into a $\sim 7^\circ$ lean from a symmetric start — the asymmetry only chose *which way*), an unexplained regression in the pitch-reflex kick-arrest failed the smoke gate, and the deployment lineage had already chosen to *randomise* the asymmetry rather than deny it (Section 3.8.12). The tool is kept; the premise — that the asymmetry was the bug — was measured to be wrong.

Mass. Every segment was weighed rather than taken from CAD:

Segment	Mass	Count
Base (torso, electronics, battery)	5764 g	1
Hip (incl. abduction and sagittal motors)	3271 g	2
Cam	66 g	2
Thigh	483 g	2
Push rod	71 g	2
Ankle spring assembly	249 g	2
Shin	324 g	2
Foot	222 g	2
Total	15 136 g	

Table 3.3: Weighed segment masses. These supersede the CAD placeholders, which totalled 12 828 g.

The correction is not small and it is not uniform. The robot is **18 % heavier** than the CAD model that every policy up to that point had been trained against, and the error is concentrated in exactly the worst place: the simulation’s shin body (shin plus ankle spring assembly, 573 g weighed) was carrying a 222 g placeholder, a factor of 2.6. Table 3.8 shows why that matters more than the total: distal leg mass is roughly nine times more costly to top speed than torso mass. The consequences of this correction are discussed in Section 3.5.

3.4 Simulation

Simulation is done in MuJoCo. The choice was forced more by the robot’s topology than by preference.

3.4.1 URDF cannot describe this robot

The parallel knee closes a kinematic loop: the cam motor drives a pushrod whose distal end ties to the shin below the knee, so hip, cam, pushrod, shin and thigh form a planar 4-bar. URDF is a *tree* format and has no way to express that closure. The exported URDF is therefore not a description of the robot but of a robot with a floating pushrod, and any simulator that consumes it directly will produce plausible and wrong dynamics.

MuJoCo’s MJCF closes the loop with an explicit `<equality><connect>` constraint. Both an open-tree URDF and a closed MJCF are exported and kept; only the MJCF is used for physics. The practical consequence for control is significant and is discussed further in Section 3.8.3: cam and thigh are *not* independent, the loop is assemblable over only a diagonal band of the nominal joint box, and commanding a target outside that band is not merely hard to track but physically impossible.

3.4.2 Compute backend

Physics runs on CPU throughout; there is no GPU physics backend in this stack. Training nonetheless runs measurably faster with the policy network on a GPU: 2900 environment-steps per second against 1530 on the same cluster node, a factor of 1.9. The cause is not matrix throughput, since the network is a two-layer MLP whose forward pass costs $117\ \mu\text{s}$ even on the robot’s Raspberry Pi. It is core contention: PyTorch on CPU steals cores from the parallel MuJoCo workers, and moving the network off the CPU returns those cores to physics.

3.4.3 How do you know a simulation result is real?

The same discipline that the reward function required (see Section 3.8.2, where the policy exploited every loophole it was offered) turns out to be necessary for the *physics* as well. A simulation bug does not usually announce itself: it produces a clean, converged, entirely plausible learning curve for a robot that does not exist. Four examples, all found while setting up a single ankle study, each of which would have silently produced a wrong answer:

- **Joint limits are not rigid by default.** MuJoCo’s default limit softness let body weight push 3.4° past the stop of a supposedly rigid strut. That is a rubber bar, not a rigid one, and it would have quietly given the rigid arm of the study a compliant ankle. Stiffening the constraint reduced the penetration to 0.08° .
- **A constraint that ships inactive is inactive.** The rigid arm re-settled its stance with a *free* ankle, because the locking equalities are only switched on at episode reset. It was being initialised in a pose its episodes never visit, 268 mm lower than intended.
- **A safety criterion measured in the wrong frame.** The workspace termination measures the toe in the *base* frame, so it cannot distinguish a collapsed ankle from a folded 4-bar. One arm stood outside the threshold from its own reset pose, and every episode was terminated 0.1 s in. The criterion had to be re-referenced per arm to its own settled stance.
- **A stale keyframe.** The shipped stance is an equilibrium of one particular ankle law. Arms with a softer ankle start off equilibrium and lurch at every reset.

A fifth class is environmental rather than physical: the local development machine ran MuJoCo

3.7.0 and the cluster 3.3.7, and a weld constraint that held on the newer solver *gave way* on the older one, drifting 0.86 rad over a 60-step rollout. The “rigid” arm was silently compliant on the cluster only. Anything solver-marginal has to be tuned against the older version.

The countermeasures are structural rather than heroic: a smoke-test suite that builds and briefly runs all 123 training presets before any long launch, self-tests inside the analysis tools that check analytic Jacobians against finite differences and loop closure against the model’s own keyframe, and assertions that fail loudly when a plant does not match what a preset claims it is.

3.5 Plant identification: replacing assumptions with measurements

Everything described so far, and every training result in Section 3.8, was obtained against a model whose parameters came from CAD and datasheets. Between 2026-08-04 and 2026-08-06 those parameters were measured on the assembled robot. This section reports what changed.

The summary is Table 3.4. It is the most important table in this chapter, and the reason is not any individual row: **every correction points the same way**. The real robot is heavier, weaker, slower and softer than the model it was trained against. That is a finding about how simulation-first development fails, not an apology for it.

Quantity	Assumed	Measured	Consequence
Total mass	12.83 kg (CAD)	15.136 kg weighed; shin body 2.6×	every policy to date trained on a robot that does not exist
Peak joint torque	170 N·m (datasheet)	144.5 N·m ($K_t \approx 85\%$)	simulation $\approx 18\%$ stronger than hardware
Drive position loop	≈ 13 Hz servo	0.8 Hz + 25 ms delay	30 % of commanded amplitude, 94° late, at 2.5 Hz
Ankle spring	$k = 28.65, 12.8$ N·m preload	$k = 41.4$, no preload	145 mm of sag; fails statics at 0.5 BW
Control loop	200 Hz, ms-scale jitter	200.0 Hz, 0.14 ms sd, 50 % of budget	200 Hz confirmed; 50 Hz is measurably worse
CAN load	assumed near the limit	26 % of one 1 Mbit bus	the bus was never the constraint
Ground friction	$\mu \approx 1$	40° slip angle, $\mu = 0.84$	train deliberately at 0.35–0.60

Table 3.4: Assumed versus measured plant parameters. Every row moved in the pessimistic direction.

3.5.1 Mass

The weighed masses of Table 3.3 were baked into the simulation model, replacing the CAD placeholders, with the inertia tensors scaled by the per-body mass ratio and the stance keyframe re-settled under the new load.

The decision worth recording is that this was *not* made an experimental variable. An earlier arrangement kept both a CAD plant and a measured plant as selectable model files, and the effect was that every preset which had not explicitly opted in was still training on the 12.83 kg robot, which was all of them but two. Splitting the plant in two had made the wrong one the default. There is one robot; it weighs 15.14 kg; the model file says so.

The accepted cost is that warm-starting any new run from a pre-correction checkpoint is now a cross-plant transfer: those weights and their observation normalisation statistics were fitted to a robot 18 % lighter with a shin 2.6 times too light.

3.5.2 Peak torque is what the gearbox delivers, not what the datasheet quotes

The actuator force ranges in the model carried the datasheet peak torque, which is the torque at an ideal torque constant. The gearbox does not deliver it.

K_t was measured by clamping a known 1.552 kg mass to the shin and repeating a quasi-static sweep with and without it. The method matters: fitting the two runs *together* with an indicator column makes the mass term’s coefficient exactly $1/K_t$, because a known mass on a known kinematic lever arm involves no link centre of mass anywhere. That breaks a degeneracy which had defeated the identification until then, since K_t and the gravity-model gain are otherwise perfectly confounded and only their product is observable.

The effective joint K_t came out at 82–89 % of datasheet, consistent with a gearbox at roughly 85 % efficiency. Peak torque scales with K_t at the same peak current, so the model was updated: $170 \rightarrow 144.5$ N·m for the cam and thigh actuators (measured), and $72 \rightarrow 61.2$ N·m for the hip-roll actuator. The second figure is the same efficiency *transferred*, not measured, because no datasheet K_t is published for that part to fit against; it is probably optimistic, since a 39:1 reduction needs more gear stages than an 8:1.

The remaining uncertainty is geometric rather than physical. The mass was clamped by hand, K_t trades against its position at about 1.9 %/cm, and the perpendicular standoff of the clamp is known only to 5–10 cm. Across that box K_t spans 82–98 % of datasheet. Settling it needs the mass on the shin centreline at a measured distance, not more analysis of the existing data.

3.5.3 The drive’s position loop is 0.8 Hz

This is the single largest modelling error found, and it was invisible until a swept-sine excitation was run on the assembled robot.

Frequency (Hz)	0.8	1.1	1.4	2.0	2.5	2.9
Gain	0.71	0.60	0.53	0.38	0.30	0.28
Phase (deg)	−48	−61	−71	−86	−94	−104

Table 3.5: Measured closed-loop response of the drive’s internal position loop, identical on both legs to two decimals. A gain of 0.71 at -48° is a first-order corner, so a single pole fits it: 0.8 Hz, plus about 25 ms of transport delay.

The simulation models these actuators as position servos with $k_p = 200$, $k_v = 5$, which is critically damped at roughly 13 Hz. The real drive rolls off at 0.8 Hz. That is a bandwidth mismatch of more than an order of magnitude, against a learnable gait frequency range whose top is 4 Hz. At 2.5 Hz the hardware executes 30 % of the commanded amplitude, 94° late. **Every policy in Table 3.11 assumes authority the hardware cannot deliver.**

Three consequences follow.

First, the un-executed part of a command *is* the tracking error, and it tends to the full commanded amplitude as frequency rises. Any high-frequency content therefore caps the usable excitation amplitude at around 15° whatever the workspace allows, which is why an early identification sweep tripped the safety limit six seconds in on the joint with the most room. Excitation

amplitude now solves against travel, no-load speed and tracking error together.

Second, **inertia cannot be excited in position mode at all**. Obtaining real joint accelerations needs either negligible amplitude at high frequency or current-mode excitation. The quasi-static runs are the valuable data and always were.

Third, and most consequential for the project, the fix is architectural rather than a gain change: the board’s cascaded position loop is proportional-only and nothing in the runtime can write its gains. The route the measurement points to is current-mode commands with the position PD closed on the Pi at 200 Hz, and a torque-source actuator with a proper torque–speed envelope in simulation instead of a fictional stiff servo. Section 3.8.1 discusses why this partially reverses a decision taken at the start of the project.

Adapting an existing policy to the measured drive cannot be done by switching it on. Applying 0.8 Hz in one step to a converged walking policy collapsed it from an episode length of 1448 and a return of +511 to 251 and –78, with nothing else changed. It is now ramped in as a curriculum, logarithmically in frequency, because the equivalent filter coefficient is strongly nonlinear in bandwidth and a linear-in-Hz ramp would do almost nothing for most of its length and then fall off a cliff.

3.5.4 The ankle spring

The fabricated spring is $k = 41.4 \text{ N}\cdot\text{m}/\text{rad}$ with **no preload**, against a model that assumed $k = 28.65$ with $12.8 \text{ N}\cdot\text{m}$ of preload at stance. Its consequences are treated in Section 3.8.5, because they overturn a conclusion drawn from the reinforcement-learning experiments.

3.5.5 System identification: including what it could not resolve

Beyond the individual parameters above, a full dynamic-parameter identification was attempted, and its limits are as informative as its results.

Method. Inverse dynamics through the closed loop was rejected: the 4-bar is near-singular over part of the operating region and the resulting inverse blows up. Identification is instead a per-joint linear regression of measured current against a gravity-torque model, viscous and Coulomb friction, an inertia term and an intercept, with the loop closure differentiated out through a constrained Jacobian.

Four estimator bugs, each of which produced a confident wrong answer. A missing sign convention on the current column made every K_t come out negative. The joint mapping never mirrored the right leg, which put the reconstructed configuration outside the assemblable band and left a loop residual of 0.78 m rather than 10^{-8} ; this reproduced identically on three separate recordings, so it was clearly not a bad capture. Pooling rows from joints that a given excitation run did not move, together with a missing intercept, each independently flipped the sign of K_t , because the drive’s DC current offset landed in the gravity column. And the loop Jacobian was over-regularised: the problem is well conditioned in this pose region, and the damping alone inflated the fitted gravity gain by 20%. With the first two fixed, the two legs agree to 5–6% instead of disagreeing in sign, which is the cross-validation that says the fixes are right.

The rejected result. With K_t pinned by the known-mass test, the model still under-predicts gravity torque by a factor of 1.3 to 1.75. The obvious explanation is that the CAD centre-of-mass offsets are wrong. That hypothesis was tested and **rejected**: gravity torque is linear

in each body’s centre of mass, so it is a single linear solve, and with all 18 parameters free, no regularisation and no plausibility constraint, moving mass explains at most 11–19 % of the residual. Ten of the eighteen directions are pure null space, which is where physically absurd fitted offsets come from. The CAD centres of mass therefore stand. Per-body mass, load-dependent stiction and base tilt were tested and rejected on the same grounds. Friction of any origin was ruled out by a decisive test: friction opposes motion and therefore reverses sign with direction of travel, whereas fitting the gravity gain separately on up-strokes and down-strokes puts both on the same side of unity for all six joints. The residual is conservative, and no friction model can produce it.

What is left lies outside the span of every rigid-body inertial parameter, which points at the kinematic model rather than the mass distribution. That has not been resolved.

Why this is worth reporting. An identification that returns a number is easy to over-trust. This one returns a measured, repeatable, per-joint gravity gain that can be applied as a correction or a randomisation range without being understood, and an explicit statement that the mass distribution is not the explanation. Knowing which parameters the experiment *could not* resolve, and why, is what stops the next person fitting the same null space again.

3.5.6 Loop timing and CAN

The whole control loop was benchmarked on the robot: drain both CAN buses, run the policy forward, write six commands. Measurements were taken with the web interface running, which is the realistic case, since it consumes roughly 70 % of a core.

Rate	Mean period	Jitter (sd)	p99	Max	Budget used
50 Hz	20.148 ms	2.151 ms	22.41 ms	49.29 ms	23 %
100 Hz	10.000 ms	0.251 ms	10.40 ms	12.28 ms	37 %
200 Hz	5.000 ms	0.141 ms	5.21 ms	7.46 ms	50 %
400 Hz	2.504 ms	0.193 ms	3.24 ms	5.13 ms	73 %

Table 3.6: Measured closed-loop timing on the robot’s Raspberry Pi 3B, with the web interface competing for CPU.

Three results, all of which contradict assumptions made earlier in the project.

The drives broadcast their state at exactly 200.0 Hz, unprompted. All six motors on both buses, period 5.00 ms, p99 5.03 ms. It is free-running and continues with the control software stopped, so it is not poll-triggered. The sensing ceiling is therefore exactly 200 Hz and matches the control stack. A 50 Hz loop would discard three of every four sensor frames.

200 Hz is clean and 50 Hz is reproducibly worse. The 50 Hz row above is not frame loss, since the interface counters show nothing dropped; it is a scheduling artefact of the longer sleep combined with a drain three times more expensive because four times as many frames have accumulated. The case for dropping to 50 Hz had been compute headroom, and compute is not the constraint: policy inference is 117 μ s mean and 144 μ s at the 99th percentile, 2.3 % of a 5 ms budget.

The CAN bus was never near its limit. Six motors at command plus reply is about 26 % of one 1 Mbit bus at 200 Hz, across two buses.

The conclusion is to deploy at 200 Hz, and there is no 50 Hz fallback worth having. Note carefully that none of this helps with the real bottleneck: running the outer loop at 200 Hz cannot

compensate for an actuator that executes a quarter of the command 94° late (Section 3.5.3). What the measurement does establish is that the Pi has ample budget to close the position loop itself, which is the proposed fix.

3.5.7 The CAN protocol, as measured

The protocol was sniffed on both live buses rather than read from documentation, and the difference mattered. Exactly two frame types exist per motor. Outbound is the command frame. Inbound is a single feedback frame at command byte `0x29`, at exactly 200 Hz, using all eight bytes: position, speed, current, temperature and an error flag.

Absent are the VESC-style extended status frames that would report input voltage and duty cycle. The hypothesis that these drives are a VESC derivative with the extra status packets available is measurably false: the command byte is `0x29` rather than VESC’s 9, so the vendor renumbered rather than inheriting. This closes the electrical route to K_t , which is why the known-mass test of Section 3.5 was necessary, and it means pack sag cannot be observed from the drives.

Worth recording as a process note: nothing in the codebase decoded the command byte, because every consumer masked it off and keyed on the node id. A comment asserting the wrong command number sat in the repository unnoticed for months, in dead code. Assumptions that are never exercised are never falsified.

3.6 Achievable gait: actuator and mass-distribution limits

The maximum running speed achievable by the robot was estimated independently of reinforcement learning using a first-principles analysis of the mechanical system. While RL demonstrates a lower bound on performance, this analysis provides an upper bound imposed by the robot’s kinematics, dynamics, and actuator capabilities.

3.6.1 Method

The analysis was performed directly on the MuJoCo model (`training/model/dash01.xml`), ensuring consistency with the model used for policy training. Candidate gaits were parameterized by duty factor D , contact length L_c , ride height h , and cadence.

For each candidate gait, the corresponding stride length, flight time, swing duration, and ground reaction forces were computed using a simplified running model. Joint torques and velocities were then obtained through the leg Jacobian throughout the gait cycle.

A gait was considered feasible only if all joints remained within the admissible 4-bar linkage assembly region while respecting actuator torque, speed, and power limits. The parameter space was exhaustively explored to determine the maximum feasible running speed.

The implementation was validated against MuJoCo. Static equilibrium predictions agreed within 1%, while the analytical Jacobian matched `mj_jacGeom` with an error below 10^{-10} .

3.6.2 Results

Regime	Top speed	Limiting constraint
Continuous operation	1.8 m/s	Continuous actuator torque (55 N·m)
Cadence ≤ 6 steps/s	3.7 m/s	Cam actuator peak power
Cadence ≤ 8 steps/s	4.4 m/s	Cam actuator peak power
Unconstrained optimum	6.9 m/s	Cam actuator peak power

Table 3.7: Maximum running speed predicted from the robot model assuming a 350 N·m/rad ankle stiffness. Only the first regime is thermally sustainable; higher speeds correspond to burst operation. **Plant epoch: CAD masses (12.83 kg), datasheet peak torque (170 N·m).** Both are now known to be wrong; see Section 3.6.6.

At the practical optimum (≤ 6 steps/s), the predicted gait has a duty factor of approximately 0.45, a stride length of 1.1–1.3 m, a contact length of 0.25–0.30 m, and peak vertical ground reaction forces of approximately 3.5 body weights (440 N), with ground contact durations between 36 and 81 ms.

The predicted continuous-speed limit (1.8 m/s) is already lower than the 2.6 m/s achieved by the RL policy, indicating that the learned controller relies on burst actuator capability. Consequently, motor temperature must be experimentally validated before sustained hardware operation. On the corrected plant of Section 3.6.6 the margin disappears entirely.

The unconstrained optimum occurs at 12.4 steps/s, corresponding to the same high-cadence solution repeatedly discovered by reinforcement learning. This agreement confirms that the high-cadence behavior originates from the system dynamics rather than from a simulator artifact.

3.6.3 Limiting factors

Maximum speed is primarily limited by swing-leg actuator power rather than by kinematic reach.

The dominant constraint is the cam actuator during swing, which requires approximately 878 W (64 N·m at 13.7 rad/s), close to its theoretical maximum output of 935 W. Consequently, increasing actuator no-load speed would improve achievable running speed more effectively than increasing peak torque.

Stance loading represents the second limiting factor. Peak ground reaction forces require approximately 167 N·m of the available 170 N·m, leaving minimal torque margin.

Kinematic reach is not the limiting factor. Although the leg can theoretically achieve a fore-aft workspace of 1.06 m, only 0.20–0.35 m is practically usable. Near the edge of the assembly region, the transmission ratio of the 4-bar mechanism deteriorates, causing joint velocities to increase rapidly and reducing achievable running speed.

Introducing a flight phase substantially improves performance. Reducing the duty factor from 0.85 to 0.45 increases the predicted speed from 4.6 to 6.9 m/s while requiring only a 1.2 cm vertical center-of-mass excursion. Lower duty factors require ground reaction forces exceeding 4.5 body weights and are therefore infeasible.

3.6.4 Sensitivity to mass distribution

Added mass location	dv/dm
Uniform scaling	-0.14 (m/s)/kg
Torso payload	-0.11 (m/s)/kg
Entire leg	-0.43 (m/s)/kg
Distal leg only	-0.93 (m/s)/kg

Table 3.8: Sensitivity of predicted maximum speed to additional mass.

Running speed is considerably more sensitive to distal leg mass than to torso mass. Additional mass located in the feet, shank, or distal linkage reduces performance by nearly an order of magnitude more than an equivalent torso payload.

For the current design, removing 0.22 kg from each distal leg increases predicted top speed by approximately 0.5 m/s, whereas adding 5 kg to the torso produces a comparable reduction. This validates the heuristic used in the design of the leg that inertia should be minimized.

3.6.5 Ankle stiffness

A static load analysis independently reproduced the ankle stiffness threshold previously observed during reinforcement learning.

The passive ankle spring (28.65 N·m/rad) supports only approximately half body weight before collapsing due to positive mechanical feedback, despite the actuators operating well below their torque limits. Increasing stiffness progressively improves load capacity, with approximately 350 N·m/rad required to support two body weights and maintain stable stance.

The agreement between the analytical load model and the reinforcement learning experiments provides independent validation of this stiffness threshold. A separate static end-effector force map (Section 3.11.1) reproduces the same conclusion from a third direction.

3.6.6 The corrected ceiling

The figures above were computed at 12.83 kg with a 170 N·m peak torque. Section 3.5 shows both to be wrong, and wrong in the direction that hurts most:

- Peak torque falls from 170 to 144.5 N·m. Since 87 % of feasible gaits are limited at the cam actuator’s peak-power corner, this is a direct hit on the binding constraint.
- Distal leg mass rises by roughly 0.70 kg across both legs (the shin body from 222 to 573 g), at a sensitivity of -0.93 (m/s)/kg from Table 3.8.

Propagating both through the sensitivity figures gives approximately **2.7 m/s credible burst and 1.3 m/s thermally sustained**, against 3.7 and 1.8 on the CAD plant. This changes the headline comparison of the whole project. The learned policy’s 2.6 m/s is no longer comfortably inside the envelope; it sits essentially *at* it, and roughly twice the sustained figure. For context, the bipedal 100 m record is approximately 4 m/s average, and a 100 m dash is about 25 s of continuous running against a continuous torque rating, so it is the *sustained* number that decides whether a record is reachable.

The honest conclusion is that this chassis is a walking and jogging demonstrator with an unusually good measurement pedigree, not a record contender. The design implications for a successor

fall directly out of the analysis above: bus voltage first, since speed scales linearly with no-load speed and not at all with peak torque, and distal mass second.

These corrected figures are a propagation of the sensitivity coefficients, not a re-run of the full feasibility sweep. The sweep should be repeated properly on the corrected plant before submission; the tooling to do so now exists in validated form.

3.6.7 Limitations

The analysis provides an optimistic upper bound on performance. It neglects torso attitude stabilization, lateral dynamics, impact losses, control bandwidth limitations, and the coupled actuator torque-speed envelope. The last of these is no longer a small omission: Section 3.5.3 measures the drive’s position loop at 0.8 Hz, so the assumption that a commanded joint trajectory is actually executed fails well below the cadences this analysis assumes.

Inertia tensors also remain approximate. The segment *masses* are now weighed (Table 3.3) and the model’s tensors were rescaled by the per-body mass ratio, but the tensor *shapes* are still CAD, and the identification of Section 3.5.5 was unable to resolve the mass distribution any further. Since Table 3.8 shows the result is most sensitive to exactly that quantity, this is the largest remaining uncertainty in the analysis.

3.7 Toy problem: one joint, end to end

Before committing a 12-DOF robot to an unproven pipeline, the complete chain — simulate, train, export, deploy on embedded Linux, close a loop over CAN — was exercised once at minimum stakes: a single-motor inverted pendulum.

The environment is a MuJoCo pendulum wrapped in a Gymnasium interface whose observation is deliberately *the hardware telemetry and nothing else*: [position (turns), velocity (turns/s), torque (N·m)], exactly as the drive reports it. The policy outputs a torque command clipped to ± 1 N·m; the success criterion is reaching the upright band within 5s and holding ± 0.1 turn and ± 0.1 turn/s to the end of a 20s episode. Training is SB3 PPO with a curriculum; the export bakes the observation normalisation statistics *into* the ONNX graph (a detail that returns as a hard requirement in Section 3.8.9); and the policy runs on a Raspberry Pi with `onnxruntime`, closing the loop on a moteus drive over CAN at 50 Hz with a watchdog. The sim-to-real transfer succeeded, and the exercise bought exactly what it was designed to buy:

- the **train→ONNX→embedded-inference path** existed and was debugged a year before the biped needed it (transport initialisation on the CAN adapter was already non-trivial and cost real iterations at this stage — cheap here, expensive later);
- the discipline of **building the observation from raw hardware telemetry** rather than simulator conveniences, which later reappears as the “no privileged observations” rule of Section 3.8.12;
- early evidence on the **compute question** of Section 3.2.3 — the BeagleBone-class board originally envisaged was rejected partly on the experience gained here.

The rig also survives as a candidate student lab exercise (TP), since it demonstrates the entire RL-for-hardware loop with one motor and one afternoon of training.

3.8 RL policy design and validation

The control policy went through three distinct action-space designs over the project. This section documents all three, the reflex structure that ended up wrapped around the final one, and the

results obtained so far. Throughout, the learning algorithm itself was held fixed : PPO from Stable-Baselines3, an actor-critic MLP, trained on CPU-parallel MuJoCo environments. So the comparisons are about the *action representation*, not about the optimiser.

3.8.1 Attempt 1: direct joint torque

The first framework commanded joint torque directly: the policy output was mapped to motor setpoints through a fixed torque scale on the actuators. This work was attempted with a simplified bipedal robot as the one developed in this thesis was still in development. This is the textbook formulation and the one most legged-locomotion papers use in simulation.

But, it was abandoned for two reasons:

It did not converge. The training history from that period is an unbroken sequence of reward-shaping attempts : raising the fall penalty because the robot learned to belly-flop, adjusting the alive bonus because it learned to dawdle, re-tuning the feet-air-time thresholds because it learned to vibrate in place. After a few iteration of this whicked whack-a-mole exploit patching, It was realised that with a raw torque action space the policy has to synthesise, from scratch and at every control step, both the gravity compensation that keeps the robot standing *and* the periodic structure that makes it walk; the reward landscape in between is dominated by ways of falling over that tricked the reward into thinking it was doing well.

More decisively, it was judged undeployable. The CubeMars drives expose a servo mode with an internal position/velocity loop running on the drive itself; commanding torque means bypassing that loop and streaming current setpoints over CAN from Python on a Raspberry Pi. The budget was estimated rather than measured, and the estimate was pessimistic on three counts: that the achievable loop rate would be low and jittery, that the CAN bus would not carry the traffic needed to go faster, and that a missed deadline in torque mode leaves the drive holding the last commanded torque blindly, which on a loaded leg integrates into a runaway rather than a pause. Only the third of those is still true. On top of it, the output-side torque constant of the actuator was not known, so the Newton-metres being commanded would have been approximate in any case.

The two arguments pointed the same way: the policy class that most needs high control bandwidth and an accurate actuator model appeared to be exactly the one for which we had neither. Torque control was dropped, and position control adopted in its place.

The measurement campaign partially reverses this decision. It is worth being explicit that this is one of the clearest results of the project, because it is a case of instrumentation overturning a design decision that had already been acted on. Every quantitative premise of the bandwidth argument was measured a year later (Section 3.5.6), and both of the first two were wrong, in the same direction:

- The Raspberry Pi holds a full 200 Hz control loop — drain both CAN buses, run the policy forward, write six commands — at a jitter of 0.141 ms standard deviation and **half its time budget spare**, with the web interface competing for CPU. 400 Hz also holds. Policy inference is 117 μ s, 2.3 % of a 5 ms budget. Far from being marginal, 50 Hz is reproducibly *worse* than 200 Hz.
- The CAN bus is at 26 % of one 1 Mbit line, and the drives already broadcast their state at exactly 200.0 Hz whether asked to or not.

So the compute and the bus, the two things assumed to be the constraint, have ample headroom. The actual bottleneck turned out to be *inside the servo mode that was kept instead*: the drive's own position loop rolls off at 0.8 Hz (Section 3.5.3), which is the single largest sim-to-real gap in

the project.

The current direction is therefore `SET_CURRENT` commands with the position PD closed on the Pi at 200 Hz, and a torque-source actuator with a real torque-speed envelope in simulation. That is the Cheetah-lineage standard, the drives support it, and the timing measurement shows the Pi has the budget to do it. It is not a full return to Attempt 1: the policy would still emit position targets, and the PD loop closing them would simply move from the drive to the Pi, where its bandwidth is known and its gains are writable. What is reversed is the premise, not the action space.

3.8.2 Attempt 2: joint position targets (PD)

The second design made the action six *position* targets, one per actuated joint, at 50 Hz, with the knee and ankle following passively through the parallel linkage and the ankle spring. In simulation these are MuJoCo `<position>` actuators with gains and force ranges taken from the datasheets of the cubemars actuator (kp 120, kv 4, 72 N·m at the hip-roll; kp 200, kv 5, 170 N·m at cam and thigh). On hardware this maps one-to-one onto what the drives actually accept.

What worked. Standing and balancing converged after a fair ammount of effort and reward shapping. After switching to a point-foot contact model only a 2.5 cm sphere at each toe collides, so the policy cannot rest a shin on the ground and damage the real robot, the policy held full-length episodes on two toe points. This also surfaced a floor-penetration exploit: with only the toe sphere colliding, the non-colliding foot and shin meshes could pass *through* the floor, and the first apparently-perfect balancing policy was largely exploiting that by slipping trough a stable equilibrium by lowering its CoM below the floor. It was fixed by terminating on both an upper-hemisphere toe contact (the sphere sank) and any foot/shin vertex below the floor plane.

What failed: skating. Walking did not. After 32 million steps the policy reached a perfectly healthy-looking reward curve and was *sliding*, not stepping. Both feet grounded on 57–66 % of steps, duty factor 0.74–0.83, median “swing” duration of one control step (contact chatter at 5–10 touchdowns per second), in-contact toe slip of 0.11–0.24 m/s accounting for essentially all of its translation and a mean forward velocity of -0.09 m/s under a $+0.5$ m/s command.

Three separate causes were found and fixed: there was no price on foot motion while in contact (a slip penalty was added, with a grounded predicate that survives micro-hop and ghost-drag dodges); the air-time term could only ever punish stepping, since credit was paid at touchdown and never lifting scored zero (it was made one-sided and capped, and paired with a per-foot stance-time cap and a swing-clearance term to give PPO an immediate gradient toward lifting a foot); and the contact physics silently made skating free, because `condim=3` disables torsional and rolling friction, turning the toe spheres into ball casters. Two reward-plumbing bugs compounded it: observation normalisation was also normalising the *reward*, dividing the carefully balanced raw scale by a return standard deviation of ≈ 110 so that a 200-point fall penalty reached the optimiser as -1.8 ; and the entropy coefficient on a clipped Gaussian policy was effectively free entropy, so the action standard deviation ratcheted up until training behaviour and deterministic evaluation were different controllers.

The lasting methodological lesson is the first one: **the system will happily exploit any loophole in the reward function.**

Why we moved on. Even after the fixes, the fundamental problem remained: a per-step position policy must rediscover the periodicity of a gait from scratch at every control step. Nothing in the representation says that locomotion is cyclic. Compounding this, I updated the

the six motor masses (7.1 kg, more than half the robot) and re-settling the stance keyframe under gravity moved the plant enough that the earlier standing policy no longer transferred, and the walking policy that had been slowly improving stopped improving at all. Rather than re-tune the same representation against a heavier plant, the representation was changed.

3.8.3 Attempt 3: a central pattern generator tuned by the policy

The third design puts the periodicity into the action space itself. Instead of emitting joint targets, the policy emits the *coefficients of a Fourier series* describing one gait cycle, and the environment reconstructs the joint targets from those coefficients at the current gait phase. This is a central pattern generator whose shape is learned rather than hand-authored: the network no longer has to invent periodicity, only to choose which periodic signal it wants. The work of the biorob was discovered after this idea emerged.

Formulation. For the two sagittal joints of the left leg (cam and thigh, the pair that drives the parallel 4-bar and therefore propulsion), the deviation from the nominal stance posture at gait phase φ is

$$\Delta q_j(\varphi) = A_j \left[w_0 a_0 + \sum_{k=1}^N w_k (a_k \cos k\varphi + b_k \sin k\varphi) \right], \quad w_k \propto \frac{1}{\sqrt{k}}, \quad (3.1)$$

with $N = 3$ harmonics, so seven coefficients per joint. The harmonic weights w_k are normalised to sum to one, which bounds the waveform, and the $1/\sqrt{k}$ schedule was chosen deliberately over the more obvious $1/k$: a $1/k$ schedule biases the series so strongly toward its fundamental that every gait it can express is near-sinusoidal, i.e. walking, whereas the push-off of a running stride is impulsive and needs high-harmonic authority. The result is clipped to $\pm A_j$ of the nominal posture a *relative* bound, because the nominal is always a validated, assemblable posture and the coupled 4-bar is only assemblable over a diagonal band of the joint box.

The phase φ advances at a frequency that is itself part of the action, so the policy chooses its own cadence. The right leg is not emitted at all: it is constructed as the mirror of the left, half a cycle out of phase, $\Delta q_{R,j}(\varphi) = -\Delta q_{L,j}(\varphi + \pi)$. This halves the action dimension, guarantees an alternating gait by construction, and matches the convention already used by the hand-authored gait demo. It has one consequence worth flagging, because it cost us time later: a mirror-symmetric gait can only travel in a straight line, so steering had to be added afterwards as two explicit asymmetry channels (differential stride amplitude and differential stance width) rather than emerging on its own.

First version: per-cycle updates, and why it was not enough. The initial implementation made one environment step equal one full gait cycle: the policy chose a gait, the environment ran it to completion (roughly 30 decisions per episode instead of 1000), and the macro reward was the mean of the per-step rewards. This was immediately better than the per-step PD policy — a hand-crafted elliptical foot path pushed through the same reconstruction walked at 0.357 m/s open-loop, versus the PD policy’s 0.01 m/s, and the first trained policy walked properly at ≈ 0.5 m/s with no skating signature at all. The structural prior worked exactly as intended.

But it was slow, and it fell over as soon as the pitch degree of freedom was released. A literature review at that point returned an unambiguous verdict: **no published system achieves running with per-cycle-only parameter updates and no intra-cycle feedback**. Every successful periodic parameterisation keeps a per-step channel — PMTG modulates the trajectory generator and adds per-step residual corrections; CPG-RL re-selects the oscillator parameters at 100 Hz. The same review found a genuine semi-Markov bug in our formulation: taking the *mean* of the

per-step rewards over a macro-step, with a fixed discount per macro-step, makes the total return scale with cadence and makes the effective horizon vary between 4.8s and 28s depending on the gait frequency the policy happened to choose. It also flagged two things the observation was missing — base velocity (the quantity the reward maximises was not observable at all) and distance-to-the-line (so the value function was partially observed and braking was unplannable).

Final version: per-step re-parameterisation plus residuals. The current stack keeps the Fourier gait but re-emits the entire specification at *every* control step, and adds a per-step correction channel on top. The 24-dimensional action is:

dims	channel	role
14	Fourier coefficients	cam + thigh gait shape, Equation (3.1); mirrored to the right leg
1	gait frequency	cadence, 0.5–4.0 Hz; the phase integrates it
3	abduction reflex gains	k_p , k_d , bias of the lateral balance reflex
6	per-step residuals	direct corrections on the six PD targets, ± 0.08 rad

Table 3.9: The per-step action. The first 18 dimensions are the gait *specification*; the last six are the fast-feedback channel.

Rewriting the gait specification mid-cycle is legal but priced: a penalty proportional to $\sin^2(\varphi/2)$ bills changes to the specification, so a rewrite is free exactly at the cycle boundary and most expensive mid-stride. The residuals are deliberately *never* billed by it, because they exist to be per-step — a one-off stumble correction should not have to pay for rewriting the whole gait. This gives a clean separation: the Fourier channel is the gait, the residual channel is the correction, and a separate magnitude penalty keeps the residual from quietly becoming the controller.

The observation is 55 values stacked over 5 frames (275 total): motor position, velocity and torque; IMU gravity direction and gyro; base velocity; $\sin \varphi$ and $\cos \varphi$; a task channel carrying the run/stop flag and the distance remaining; and the previous action.

Reward. The sprint objective is a single dash — stand at the line, run, stop. The core insight, worth stating because it is not obvious, is that **a per-step speed reward cannot price time**: integrating $w v_x$ over a fixed course gives $w \cdot \text{distance}$ regardless of pace, identical for a fast run and a slow one. What prices time is a *constant per-step clock cost*, which integrates to $-w_t T$; and correspondingly the alive bonus must be exactly zero, because paying per second of existence rewards dawdling. The break-even pace is the ratio of the two, 0.25 m/s.

Around that core sit the anti-skate terms inherited from the PD era, a Siekmann-style phase-gated contact schedule (each foot pays for ground contact during its expected swing window, using the action’s own gait clock; as the stance ratio ramps from 0.65 down to 0.42 the two swing windows begin to overlap, so a flight phase is explicitly demanded — this is the one term that asks for running rather than fast walking), and efficiency terms on torque, motor velocity and positive mechanical power, ramped in over 60 million steps so they cannot smother gait emergence early.

Four reward exploits were found by adversarial review of the reward code and fixed before the long runs: a stop-phase income that was net-positive per step and therefore farmable forever by twitching the hold timer; a one-sided velocity clip that made shuttling back and forth in front of the finish line strictly better than crossing it; and a cold-start suicide, where the sum of always-on penalties for a standing robot (≈ -2.2 per step) made diving at -100 value-optimal for any policy that could not yet locomote — fixed by flooring the per-step total, not just the

individual terms.

Curriculum over base degrees of freedom. Rather than ask a fresh policy to solve balance and locomotion simultaneously, the base is progressively released. The free joint was replaced in the model by six explicit scalar joints (x, y, z , roll, pitch, yaw) plus six equality constraints that can be activated individually, so any subset of the base degrees of freedom can be rigidly railed at reset. Milestone **m1** frees only forward translation (with height railed at a randomised ride height drawn from a measured lookup table), **m2** adds height, **m3** pitch, **m4** lateral translation, **m5** roll, **m6** is the real plant. Rigid equality constraints were chosen over clamping the state after each step precisely because they produce the correct reaction wrench *during* the step, so the leg torques stay faithful. Critically, the observation and action dimensions do not change across milestones, so every stage warm-starts from the previous one’s checkpoint.

3.8.4 Reflexes: what is learned and what is hard-wired

Not everything in the controller is a Fourier series, and the split is deliberate. A Fourier series over gait phase is by construction *feedforward*: it is a function of where we are in the cycle, not of what the robot is doing. Balance is not a periodic problem, it is a feedback problem, and encoding it as a phase-indexed trajectory would make it blind. Three feedback channels therefore sit outside the CPG.

Lateral (abduction) reflex — learned gains, fixed structure. The hip-roll joints are not driven by the gait generator at all. They are driven by a linear reflex on base roll,

$$u = k_p \phi_{\text{roll}} + k_d \dot{\phi}_{\text{roll}} + b, \quad q_{\text{hip roll},L} = q_L^{\text{nom}} + u, \quad q_{\text{hip roll},R} = q_R^{\text{nom}} - u, \quad (3.2)$$

evaluated at every control step. Because the left and right roll axes are mirrored, opposite signs in joint space move both feet the same physical way, which is the lateral centre-of-mass shift we want. The three gains (k_p, k_d, b) are *emitted by the policy*, so the controller is learned — but only three numbers of it are, rather than a free trajectory. This was the design decision that made the original per-cycle formulation tractable at all (a feedforward CPG for propulsion plus a feedback reflex for balance, instead of a per-step policy with free abduction), and it was kept in the per-step version because it works. Nothing assumes which sign stabilises; the policy discovers that. The reflex is necessarily inert while roll is railed (milestones **m1–m3**, where the gains are unidentifiable) and only becomes live at **m5** — expected behaviour, not a bug, and worth knowing before reading those training curves.

Sagittal (pitch) reflex — hand-tuned, not learned. A fixed PD on base pitch and pitch rate drives a *symmetric* fore/aft thigh offset,

$$u_p = -\text{clip}(k_p^\theta g_x + k_d^\theta \dot{\theta} + b^\theta, \pm u_{\text{max}}), \quad q_{\text{thigh},L} += u_p, \quad q_{\text{thigh},R} -= u_p, \quad (3.3)$$

where $g_x \approx \sin \theta$ is the fore/aft component of the measured gravity direction. Again the thigh axes are mirrored, so the opposite signs move both feet the *same* physical direction: when the robot pitches nose-down, both feet shift forward, under the falling centre of mass. This is capture-point control in its crudest form, and the gain scales follow that reading ($k_p \approx h_{\text{CoM}}/\ell \approx 1$, $k_d \approx \sqrt{h/g} \approx 0.3$).

The reason this one is *not* learned is worth spelling out, because it is the opposite choice from the abduction reflex. It was introduced when the pitch-free milestone was collapsing in 1.5 s, and a learned gain has no useful gradient in that regime: the policy has to survive long enough to experience the correction before it can learn to tune it, and it was not surviving. A hand-tuned

prior breaks that circularity. The gains were set by a grid search on the pitch-free plant: $k_d \geq 0.3$ oscillates through the one-step action delay and the target smoothing filter, and an authority clip above 0.35 rad starts fighting the gait itself, which fixes $k_p = 2.0$, $k_d = 0.2$, clip 0.25 rad. Measured on a pitch-kick arrest test, the reflex reduces the final pitch excursion by a factor of 2.6–6.7 in both directions, and lifts early-retraining survival from about 18 to about 80 control steps.

That number is the point: the reflex is a **stabilising prior, not a controller**. The plant with height free cannot stand at zero action at all — holding a ride height requires an active gait, so it collapses in ≈ 1.3 s no matter what the pitch does. The reflex does not fix that; it buys the policy enough surviving episodes to find a pitch-neutral gait itself. An early acceptance gate of “zero action stands for 20 seconds” was simply wrong for this plant and was replaced by the kick-arrest test.

Residuals — the fast channel. The six per-step residuals are the third feedback path, and the one the literature is most emphatic about: they are what lets the controller respond to something unmodelled within a single control step without rewriting the gait. They are kept small (0.08 rad, raised to 0.20 rad once the control rate moved to 200 Hz) and are magnitude-penalised, so the Fourier prior stays dominant.

All three are implemented in pure NumPy, in the same module that reconstructs the gait, with no dependency on the simulator. That is not incidental — it is what allows the identical reflex code to run on the Raspberry Pi against a real IMU, rather than being re-implemented and re-tuned at deployment time.

A negative result: the emulated ankle reflex. One more reflex was built and rejected. Since the ankles are passive springs with no actuator, we implemented a fixed PD that applies a pitch-restoring *torque* at the ankle joints, gated on that foot being grounded — an emulated actuated ankle, and a shippable controller if a real ankle motor were ever added. At 40 N·m of authority it *destabilised* the running gait outright (episodes collapsed to 28 steps): a phase-blind torque fights the gait it is trying to stabilise. At a gentle 12 N·m it neither destabilised nor solved anything, giving a small, non-solving lift. The negative result was still informative — it was evidence that ankle authority was the right lever, which is what motivated the passive-stiffness experiment described next.

3.8.5 The pitch-balance wall, and how it was broken

Everything through milestone m2 (forward translation and height free, attitude railed) trained well. Releasing pitch at m3 produced the hardest failure of the project: episodes plateaued at roughly one second — the passive fall time — and stayed there. Two full sessions of work went into it, and it is worth recording what was ruled out, because the null results are the useful part.

What was tried	Outcome
Reward/curriculum easing (7 variants: no flight demand, efficiency off, low speed cap, more authority, height shaping, competence-gated hardening)	All plateaued at the same episode length. The reward was not the bottleneck.
Decaying external pitch assist (a spring-damper on the base pitch joint, faded to zero)	<i>Crutch dependence.</i> In-training episode length looked excellent; with the assist forced to zero at evaluation the policy was <i>worse</i> than the plateau, and drifting backwards.
Anti-crutch penalty on the assist torque the policy provokes	No improvement at low weight, worse at high weight.
Pitch “slow motion” via extra armature on the base pitch DOF, faded out	Marginal gains that did not survive the fade.
Raising the control rate 50–200 Hz and increasing residual authority (0.2–0.4) and filter bandwidth	All plateaued at ≈ 1 s. Since the learned residual gives <i>more</i> foot-placement authority than any fixed heuristic could, this implicated the plant rather than the control law.
Emulated actuated ankle (fixed torque reflex)	Strong version destabilised; gentle version marginal.
Lowering the centre of mass by 6–12 cm	Slightly <i>worse</i> .

Table 3.10: The pitch-balance failure: everything that did not work.

Two methodological lessons came out of this table. First, **any training aid that holds the robot up will be leaned on**, and the in-training curve will look wonderful while the unaided policy gets worse; only an evaluation with the aid forced to zero tells the truth. We were misled by this three times before adopting the assist-off evaluation as protocol. Second, when every software lever saturates at the same value, the constraint is not in the software.

The diagnostic that cracked it. Rather than try a ninth lever, we instrumented the failure: a tool that rolls out a policy and plots, per foot, the fore/aft offset between the toe at touchdown and the whole-robot centre of mass. The result was unambiguous. The base pitch ramps monotonically forward into a pure topple, and meanwhile the feet land 8–9 cm *behind* the centre of mass — only 11 % of touchdowns landed ahead of it, and they fell progressively further behind as the body ran away forward. The policy had a perfectly serviceable forward-running gait and had never learned the *capture step*: plant the foot ahead of the centre of mass to catch a forward fall. It was, quite literally, running out from under itself.

The fix was a plant change. The candidate lever this pointed to was the passive ankle. Stiffening the ankle spring by roughly a factor of twelve (28.65–350 N·m/rad) gives the passive foot the stance pitch-restoring authority the plant lacked. The threshold is clean: $k \leq 200$ barely helps, $k \geq 350$ breaks out of the plateau and climbs steeply. Post-fix, the same diagnostic reports 67 % of touchdowns landing ahead of the centre of mass and the base pitch pinned near zero.

One implementation detail is essential and cost us a wasted run. The standing ankle sits at $\approx \pm 0.25$ rad while the spring reference is at ± 0.7 rad, so the spring carries ≈ 12.8 N·m of *preload* at stance. Naively raising the stiffness balloons that preload, the foot slams to its spring reference and the robot flips over. The reference has to be shifted simultaneously so that $k(q_{\text{stand}} - q_{\text{ref}})$ is preserved: only the restoring *gain* rises, the posture is unchanged.

This result also *overturned* our own earlier recommendation. After the emulated-ankle experiments we had concluded that the robot needed a policy-controlled actuated ankle — a motor, two more action dimensions, and a full retrain of the milestone ladder. The stiffness experiment shows a passive spring is sufficient, which is dramatically cheaper. And, as noted in Section 3.6, the static load analysis independently arrives at the same 350 N·m/rad threshold.

That left one question open — whether the real ankle could be built that stiff — and the answer, when it was measured, invalidated the result.

The measured spring, and what it does to this conclusion. The fabricated ankle is $k = 41.4$ N·m/rad with **no preload**. Neither number matches the model: $k = 350$ will not be built, and the 12.8 N·m of stance preload that the simulation carried does not exist on the robot.

The second discrepancy turns out to matter more than the first, which is the genuinely useful finding here. The preload, not the stiffness, was doing almost all of the work. At the same k , the modelled spring sags 19 mm at stance with its preload and **145 mm without it**. That reframes the whole ankle question: the parameter the study spent its compute sweeping was not the parameter that mattered.

Statics disposes of the real spring before any learning is involved. At 0.5 body weights per foot — double support, the easiest possible load case, against a 3.5 BW running requirement — the no-preload spring deflects 26.6° , drops the robot 145 mm, and puts the heel 11.9 mm into the floor at the reset pose. Contact forces are well behaved throughout, so this is genuine load-capacity collapse rather than a solver artefact.

Everything in Table 3.11 was therefore obtained on an ankle that does not exist. The balance result of this section is real as a statement about what the *plant* needed; it is not a statement about what this robot does.

The live candidate is now neither the fabricated spring nor a stiffer one: it is a tension-only strut, taut at the flat-foot angle and buckling in compression. It stands like a welded ankle (0.08° of deflection, 0.8 mm of sag) and, by deleting the spring assembly, removes 249 g from each shin. Table 3.8 independently favours that: at -0.93 (m/s)/kg on distal mass, 249 g per leg is worth roughly $+0.5$ m/s of top speed, which is the largest single improvement available anywhere on the robot. A design decision reached for one reason turns out to be supported by an unrelated measurement, which is the closest this project comes to a free result.

3.8.6 The cadence problem and a cautionary bug

With balance solved, the remaining pathology was cadence: the policies stepped far too fast, on the order of 11 Hz, pattering rather than striding. Several external limits were tried — hard velocity caps, acceleration caps, a cadence penalty — and all of them collapsed the warm-started policy instead of slowing it.

Instrumenting the trained policy’s own action channels found why. When the control rate was raised to 200 Hz, the learnable gait-frequency range had been widened to 0.5–50 Hz on the reasoning that it would then be “non-binding”. But the map from the raw action to frequency is linear over that range, so a *neutral* network output (≈ 0) means 25 Hz. The policy simply left that output near zero and inherited a 24 Hz gait clock — the commanded thigh trajectory reversing

sign 35 times a second at ± 0.45 rad, a signal that would destroy the real actuators. Every external limit had been fighting a broken internal default. Remapping the range to 0.5–4.0 Hz, so that a neutral action means ≈ 2 Hz, produced the best controller of the entire project (Table 3.11).

The general lesson is worth carrying: **a parameter range widened to be “non-binding” can silently become the operating point**, because a network’s default output is not a neutral gait — it is whatever the middle of the range happens to decode to.

A residual pathology survives the fix. After the remap, the gait clock slowed to ≈ 1 Hz but the actual *footfall* rate stayed near 6 Hz: the policy steps through the fast residual channel rather than through the clock. A ring-down test showed the passive ankle mode sits at 1.0 Hz, not 6.3 Hz, so this is a forced oscillation, not a resonance — the policy pumping its legs to balance a top-heavy robot at 200 Hz. Neither a residual-rate penalty, a torque-budget curriculum, reducing the pitch-reflex derivative gain, low-pass filtering the reflex input, nor heavy ankle damping removed it, and warm-started runs are all trapped in the fast-stepping basin. The honest conclusion is that it needs a from-scratch retrain with cadence pressure present from step zero, or a plant change (a lower centre of mass); on hardware, the real mechanical damping of the ankle would absorb some of it.

3.8.7 Results

Milestone	Free base DOF	Steps	Survival	Outcome
m2	x, z	180 M	53 s	runs; the reliable working regime
m3	+ pitch	600 M	39 s	completes the dash and stops, 2.6 m/s
m4	+ y	600 M	38 s	completes the dash and stops
m5	+ roll	600 M	26 s	high variance — the weak link
m6	all six	600 M	60 s	never falls; runs through the line
m7	+ pitch (cadence fix)	163 M	37 s	best controller: 2.71 m/s, very low variance

Table 3.11: Deterministic (greedy) evaluation of the milestone ladder, each stage warm-started from the previous. m7 is not a new degree of freedom but the m3 plant retrained after the gait-frequency fix of Section 3.8.6. Survival is capped at 60 s by the episode limit, so m6 is at the ceiling. **Plant epoch: CAD masses (12.83 kg), $k = 350$ N·m/rad ankle with preload, datasheet peak torque, an idealised ≈ 35 Hz drive.** None of those four match the measured robot; see Section 3.8.8.

The headline result is that **the fully free 6-DOF plant runs and does not fall** — 60 seconds of simulated running, at the episode limit, with zero variance across evaluation episodes, at 2.6 m/s. Two caveats belong next to it: m6 balances but runs *through* the finish line without executing the stop (a task-completion defect, not a balance one), and m5 still topples on a fraction of episodes and needs more training. Against the capability bound of Section 3.6, 2.6 m/s is about 70 % of the credible-cadence ceiling and already above the thermally sustainable figure. On the corrected plant of Section 3.6.6 it sits at the burst ceiling and at twice the sustained one.

An A/B on the generator itself. Because the Fourier series is one of several ways to parameterise a periodic gait, it was compared directly against a phase-amplitude coupled-oscillator generator of the kind used in the biologically-inspired locomotion literature, with the foot trajectory mapped through a measured inverse-kinematics table. Sixteen runs, one plant, one reward, one schedule, only the generator swapped. The verdict is that **there is no clear winner**: the oscillator is more sample-efficient from a cold start on the hardest milestone, the

Fourier generator is more reliable across random seeds, and the single best controller measured in the whole experiment was an oscillator run — the only arm of the sixteen that ever executed the stop phase, at 2.9 m/s with a left/right asymmetry of 0.08.

One ablation in that experiment is worth reporting because it refutes a hypothesis stated earlier in this chapter. Removing the per-step residual channel, leaving a pure oscillator, produced *stability without locomotion*: the robot survived most of the episode but travelled 15 m of 72 at 0.25 m/s, with the worst cadence in the entire set. The residual channel was the suspected fast open path responsible for the 6 Hz footfall of Section 3.8.6; removing it made the chatter worse, not better, so that explanation is wrong.

Cost. Training ran on the EPFL Izar cluster. Throughput is ≈ 2900 environment-steps per second on a V100 node, which is $\approx 1.9\times$ the CPU-only rate on the same node — and, worth recording, not because the network is large. Physics is always on the CPU (there is no GPU physics backend in this stack); putting the tiny policy network on the GPU simply stops PyTorch from stealing cores from the parallel MuJoCo workers. At that rate a 600 M-step milestone is a multi-day job, which is why the warm-start chain across milestones matters as much as it does.

3.8.8 Which of these results survive the plant correction

Section 3.5 invalidates the plant that every number in Table 3.11 was measured on. Rather than delete the table, it is more useful to state precisely which conclusions depend on the plant and which do not, because that distinction *is* the sim-to-real argument.

Survives a plant change	Does not
The action-space comparison: torque, per-step position, and periodic generator with residuals, in that order of tractability.	Every absolute speed and survival time in Table 3.11.
The reward-exploit register (Section 3.8.2) and the general lesson that reward curves cannot certify a gait.	The claim that the fully-free plant “runs and does not fall”. It ran, on a plant with an ankle that will not be built.
The base-DOF curriculum, and warm-starting across stages with fixed observation and action widths.	The ankle-stiffness recommendation, which Section 3.8.5 replaces.
The gait-frequency bug and its general lesson about non-binding parameter ranges.	The 2.6 m/s figure as evidence of hardware capability.
The capture-step diagnostic and the finding that foot placement lagged the centre of mass.	Any claim of readiness for deployment.

Table 3.12: What the plant correction of Section 3.5 does and does not invalidate.

The pattern is that the *methodological* results transfer and the *performance* results do not. That is an unsurprising conclusion in hindsight and an expensive one to learn: roughly 3 billion environment-steps of training were spent on a robot 18 % too light, 18 % too strong, with an ankle spring an order of magnitude too stiff and an actuator an order of magnitude too fast.

3.8.9 Validation status and known gaps

Stating this plainly matters more than the results above, because the gap between them is the work that remains.

Sim-to-sim is exercised continuously by the milestone ladder: each stage is a different plant (different constrained dynamics) and every stage loads the previous stage’s weights, so a policy that only worked by exploiting a particular rail fails visibly at the next rung. The ankle-stiffness and centre-of-mass experiments are the same mechanism used deliberately as a plant-sensitivity probe, and the drive-bandwidth correction of Section 3.5.3 is the most severe instance of it: a converged walker collapsed from an episode length of 1448 to 251 with only the ankle and the drive changed.

No trained policy has run on the robot. That is the precise statement, and it is narrower than “nothing has run on hardware”. A great deal has: the hand-authored fixed gait plays on the real motors, the web interface drives all six joints under safety limits, the workspace calibration and the system-identification excitation runs are all recordings taken from the machine, and every measurement in Section 3.5 came off it. What has never happened is a learned controller closing a loop on the physical robot.

The sprint policies in Table 3.11 are not deployable as they stand, independently of the plant correction. They observe two privileged quantities that do not exist on the robot: the base linear velocity, which would require a state estimator that has not been built, and the distance remaining to the finish line. A separate policy lineage was therefore started for a drivable demonstrator, which removes both — velocity has to be inferred from a 200 ms strided observation history instead — and adds per-episode plant randomisation (mass, inertia, friction, servo gains, joint damping, ankle spring stiffness, gravity tilt, actuation delay), a sensor-noise model including the accelerometer leakage that makes a real IMU’s “down” swing by several degrees during push-off, and the steering channels the mirror-symmetric gait lacks. That lineage cannot warm-start from the sprint milestones, since the observation and action dimensions both changed, so it is a separate training cost. Its results, and the diagnoses that produced them, are reported in Section 3.8.12.

Domain randomisation ranges are now set from measurement rather than from taste where the measurement exists. Ground friction is sampled over 0.35–0.60 against a measured $\mu = 0.84$ (a 40° slip angle), deliberately well below the real value, because the failure mode being guarded against is planting a foot that then slides. Joint homing error is sampled at 5° against a calibration accurate to 1–2°, as a safety envelope. Where no measurement exists the range stays wide and is flagged as such: link inertia tensors are randomised at $\pm 25\%$ precisely because Section 3.5.5 could not resolve the mass distribution.

The remaining known gaps, recorded deliberately rather than discovered later: there is no policy export path and no inference loop on the Pi, which is the critical path for everything else; and the actuator model still gives each motor an independent torque limit and speed limit rather than a coupled torque–speed envelope, which makes the simulation optimistic exactly in the high-speed regime the project cares about. First hardware bring-up remains planned with the base fixed and the legs in the air, which validates wiring, direction conventions, joint limits and gait shape, but with no ground contact tells us nothing about balance or transfer.

The five subsections that follow report what has happened since this status was written: a generator A/B (Section 3.8.10), the ankle resolved as a build decision (Section 3.8.11), the drivable lineage’s descent onto the measured plant (Section 3.8.12), a classical baseline built to test whether learning was ever the bottleneck (Section 3.8.13), and the rebuilt robot (Section 3.8.14).

3.8.10 A second generator: explicit-oscillator CPG versus the Fourier series

The Fourier parameterisation of Section 3.8.3 is one member of a family; the other canonical member is the explicit limit-cycle oscillator of the Ijspeert school, as used by CPG-RL. Since the literature gave no basis to prefer one, it was measured: sixteen cluster jobs, one plant, one reward, one schedule, *only the generator swapped*. Building the oscillator arm forced a foot-space formulation with a measured inverse-kinematics lookup table, and the LUT construction itself surfaced plant facts worth recording: from the stance pose *both* cam and thigh move the toe mostly fore-aft and either sign of cam *shortens* the near-straight leg (the intuitive “thigh swings, cam lifts” mapping is simply wrong and produces no step); the 4-bar has a folded solution branch that must be filtered; the IK inversion must propagate continuity between cells or it flips branches mid-swing; and reachable foot lift at mid-stride is only ~ 8 cm, because the leg is near-straight there.

Condition	Oscillator (CPG)	Fourier
cold m3 (from scratch, pitch free), 133 M m2 (pitch railed)	ep_len 408, return +4 best single controller of the experiment: 73.5 m dash, 2.9 m/s, <i>the only arm that executes the stop phase</i> , L/R asymmetry 0.08	ep_len 237, return -89 fastest arm of the experiment (3.30 m/s, 5/5 episodes) but never executes the stop
m2→m3 warm start	3/5 episodes cross at 2.40 m/s	5/5 cross at 2.45 m/s

Table 3.13: Generator A/B at matched budget. No clear winner: the oscillator is more sample-efficient cold, the Fourier series more reliable across seeds; both show the same bimodality on the warm-started pitch-free plant.

Two results stand out beyond the tie. First, the **pure oscillator with the residual channel removed produces stability without locomotion**: it survives 9700 of 12 000 steps but travels 15 m of 72 at 0.25 m/s with the worst foot-scutting cadence of all sixteen arms. This *refutes* the standing hypothesis that the per-step residual channel was the open fast path causing the 6 Hz footfall chatter of Section 3.8.6 — removing it made chatter worse, not better — and it is the empirical form of the literature’s verdict that a per-step channel is mandatory. Second, when the full base-DOF ladder was subsequently run on the oscillator generator, it **cleared m1–m4 (100 m dash at m2, 2.95 m/s at m3) and walled hard at m5 (roll)** — episode length 337 against the Fourier ladder’s 3213 at matched budget, with the plateau confirmed at $2.5\times$ budget. On the roll axis the Fourier generator succeeds where the oscillator does not; both share the identical 3-parameter roll reflex, so the difference is in the generator itself and points future investigation at lateral authority. The pragmatic conclusion adopted: keep the Fourier generator for the deployment lineage, use the oscillator result as the tiebreak evidence that the action-space *family* is right.

Getting this A/B to run also produced a curriculum lesson expensive enough to state as a rule. One milestone in the ladder took six restarts, five of them self-inflicted, and the terminal bug was a competence gate that held *both* the difficulty ramp and the efficiency-penalty ramp: for a policy that never reached the gate, the efficiency terms were pinned at exactly zero for the entire run — a run that looked like “the robot cannot learn this” was actually “the reward is not the one we designed”. What settled it was not another hypothesis but a field-by-field configuration diff against the last run that worked. **Never gate the ramp that is supposed to rescue competence on competence itself; and diff configurations before theorising.**

3.8.11 The ankle, revisited as a build decision

The 350 N·m/rad ankle-stiffness result of Section 3.8.5 answered a simulation question; the hardware then asked a different one. The spring actually fabricated measures $k = 41.4$ N·m/rad with *no preload* (Section 3.5.4), and a spring wound to 350 will not be manufactured. The question became: what should be *built* — the real soft spring, or no spring at all (a rigid strut), which would also delete the 249 g spring assembly from each shin, on the axis the sensitivity analysis prices at -0.93 (m/s)/kg?

The soft no-preload spring is eliminated by statics alone, before any learning. At 0.5 body weights per foot — double support with the base held, the easiest load case there is, against a 3.5 BW running requirement — the as-built spring deflects the ankle 26.6° past flat, drops the robot 145 mm, and rotates the foot so far that the heel carries 208 N with the toe driven 11.9 mm *into* the floor at the reset pose. No controller fixes this; a policy chooses where to put a foot, it cannot make a spring hold seven times its load. The instructive split is preload versus stiffness: the same $k = 41.4$ sags 19 mm *with* the original preload versus 145 mm without — **the preload, not the stiffness, was doing $7.6\times$ of the work** in the shipped ankle.

The strut wins the learning screen. In the RL comparison (cold m3, corrected plant), the compression-capable strut arm reaches episode length 2307 in *one third* the steps of the fully welded ankle (29 M vs 73 M) and walks $3\times$ more symmetrically; the welded arm, given time, completes the 57 m dash at 2.55 m/s in 5 of 6 episodes, while a tension-only strut (no compression capacity) manages 0.6 m and stands on one leg. The compression capacity is the entire story — the arms are statically identical otherwise.

And the as-built part fails by arithmetic. The fabricated strut is a 500 mm $\times$ 3 mm solid PETG rod: slenderness $L/r = 667$, Euler buckling load **0.24–0.33 N** — thirty-five times below the 10 N the simulation arm assumed, and ten times less moment at the ankle than the foot’s own gravity torque, so it cannot even hold the foot flat in swing; its tension capacity (283 N) is also below the 360 N walking peak. A 3 mm pultruded carbon rod (P_{cr} 14.1 N, 4241 N tension, 5.7 g) or a 6/4 mm carbon tube (181 N) reproduces the simulated part. **The ankle question closed as a build instruction: make it a stiff link, in carbon, and delete the spring mass.**

The instruction was executed (Section 3.8.14): the ankle now is a rigid carbon tube weighing 40 g, removing a net 209 g per shin ($573 \rightarrow 364$ g; the robot totals 14.72 kg) on the axis worth -0.93 (m/s)/kg — 209 g off each shin buys more predicted speed than 1.8 kg off the base would.

3.8.12 The drivable-demonstrator lineage: closing on the measured plant

The deployable lineage announced in Section 3.8.9 has since accumulated results, and they are the closest thing this project has to a sim-to-real verdict short of untethered hardware. The lineage removes both privileged observations (velocity must be inferred from a 200 ms strided history), adds steering, and layers on the full sensor/plant randomisation package. Its history is a sequence of measured diagnoses:

Steering was broken twice over, and both causes were in the plant model. The first policy could not turn right at all. An open-loop authority test (hand-built gait, roll and pitch railed — isolating plant from policy) found (a) the *model* is not left/right symmetric: two linkage anchor coordinates differ by ~ 1 mm in the wrong axes, a CAD-export artefact worth $+0.42$ rad/s of intrinsic yaw; and (b) the two steering channels were scaled inversely to their

measured authority — differential stance width is $\sim 6.6\times$ stronger per unit than differential stride, and the weak channel’s whole authority (0.26 rad/s) was smaller than the intrinsic bias. The fix randomises the anchor asymmetry (± 1.5 mm) rather than symmetrising the file — the real robot will be asymmetric too, and on a perfectly symmetric plant the policy has no reason to learn the correction that transfers. Post-fix, yaw-response slope went from 0.19 to 1.22 with both turn directions reachable.

Domain randomisation was the thing breaking the policy — measured, not assumed.

On a fixed 20-seeded-episode evaluation, removing one adversity at a time from a trained policy: pushes changed nothing ($1.0\times$ mean time between falls), sensor noise $1.5\times$, trips $1.8\times$ — and **plant randomisation $19.8\times$** (95.4 s vs 4.8 s MTBF; a clean plant gives 196 s and zero falls). The policy walked fine; full-width plant DR from step zero was what kept it down, while every other axis already had a curriculum. DR width now rides a gated, retreating ramp like everything else. Three evaluation-hygiene rules were paid for here: training reward across runs with different curricula is not comparable (always evaluate on a fixed task); six evaluation episodes gave a $5\times$ spread on fall rate for the *same* policy (use ≥ 20 seeded episodes); and a curriculum that can only advance will ride a policy down — competence must gate the *advance*, not just the opening (one run peaked at 113 M steps and declined monotonically for 215 M more while the clock-driven ramps kept tightening).

The measured plant, applied all at once, kills a working policy — applied as a curriculum, it does not.

With every measured correction active (weighed masses, 144.5 N·m torque, strut ankle, 0.8 Hz + 25 ms drive, friction deliberately below the measured $\mu = 0.84$), the previous best walker collapses from episode length 1448 / reward +511 to 251 / -78 with nothing else changed. Ramping the drive bandwidth logarithmically in Hz (the filter coefficient is violently nonlinear in bandwidth) over 120 M steps, the same policy completes the descent to the real 0.8 Hz drive and stabilises at **episode length 4938 (peak 6833) and reward +3271** — three times its parent’s score, *on the real drive*. That checkpoint is the warm-start parent for everything downstream. The residual gap to a deployable walker is honest but bounded: on the hardest configuration the current generation survives a median 9.4 s with 3 of 12 seeded episodes reaching the 60 s cap, walking at ~ 0.14 m/s with a limp — and the command-range fix this motivated (the command box asked for 0.6 m/s from a plant that tops out at 0.14; forward commands above 0.3 m/s are fatal while *backward* is the safest direction) is in the currently training generation.

The lineage also re-proved the ladder.

This is the one lineage in the project that trains with all six base degrees of freedom free from step zero — and the one that does not converge (three plateaus across ~ 1.4 G steps). Meanwhile the only controller to reach 2.55 m/s on the *measured* plant did so at milestone m3, roll and yaw railed, in 73 M steps. Everything that has ever worked on this robot used the base-DOF ladder; the audit of the archive (97 runs at m3) makes that an empirical claim, not a preference. Two architectural responses are now in training: restarting the lineage *through* the ladder on the rebuilt plant, and an asymmetric actor-critic — the critic keeps the privileged observation, and a separately supervised velocity-estimator head (trained on labels, detached from the policy gradient so PPO cannot bend it toward whatever the actor wishes were true) replaces the privileged velocity for the actor, motivated by the measurement that only $R^2 \approx 0.81$ of base velocity is recoverable from the observation history.

Two bugs from this lineage belong in every RL practitioner’s checklist.

A warm-started run inherited the *source checkpoint’s* RNG seed — so every warm-started “seed replicate” in the project was a bit-identical duplicate (verified: two seeds, 100 rollouts, identical curves)

until the seed was re-asserted after loading. And the evaluator rebuilt the environment from the config’s *starting* values rather than the curriculum’s final ones, reading a 340 M-step policy at $32\times$ below its true performance — the fourth evaluation-restore bug of the project, and the reason the rule “`curriculum.json` is ground truth for anything a curriculum moved” now exists. Related: outcomes on the hard plant are *bimodal* (fall at ~ 2 s or survive 60 s), so unpaired small-sample comparisons are noise — one early comparison “found” a $27\times$ sensor-noise effect that does not exist. Common random numbers and survivor counts, minimum twelve episodes, are the reporting standard for everything above.

3.8.13 A scripted baseline: how far does no learning get?

Late in the project a classical controller was written for the same plant: a clock-driven foot trajectory through the measured foot IK, once-per-swing Raibert/capture-point foot placement, low-gain pitch and roll trims, and feedforward inversion of the 0.8 Hz drive — 21 hand-set gains, zero training, injected through the environment’s residual channel so that every plant feature (drive model, delay, randomisation, contact, termination) is byte-identical to the RL runs. Its purpose was to answer a question the RL curves cannot: when training stalls, is the *learning* the bottleneck, or the machine?

The controller’s structure is itself an argument about the plant. The foot-placement target is latched *once per swing* rather than tracked continuously, because the drive is a 0.8 Hz pole ($\tau = 199$ ms) against an inverted-pendulum divergence constant of $\sqrt{h/g} = 311$ ms: a continuous pitch loop would need crossover around 1 Hz *through* a 0.8 Hz actuator pole, spending $\sim 60^\circ$ of phase before acting, whereas latching at swing entry gives the drive a whole swing ($\approx 1.8\tau$), delivering $\sim 83\%$ of the commanded placement by touchdown. The same arithmetic sets the tuned cadence near 1 Hz (at 2 Hz the drive returns 0.38 of the commanded amplitude) and motivates the one element RL structurally lacks: explicit feedforward inversion of the drive pole, $q_{\text{cmd}} = q + \tau \dot{q}$.

The answer is sharp in both directions. On the height-free, attitude-railed plant (m2), the scripted controller runs **60 s on all 5 seeds** — where the RL reference achieves 20.3 ± 21.6 s — winning less on the mean than on the variance, since the RL outcome is bimodal. On the pitch-free plant (m3) it survives **1.0 s, 0 of 5**, against the RL reference’s 1.15 s: *the same wall, to the decimal*.

The wall is free fall, proved by a formula with no controller in it. Modelling the robot as a rigid inverted pendulum about the toe, the passive topple time from initial lean θ_0 is $t = \sqrt{h/g} \cosh^{-1}(60^\circ/\theta_0)$. Sweeping gravity in simulation:

g (m/s ²)	9.81	6.00	3.50	2.00
predicted topple time	0.93 s	1.19 s	1.56 s	2.06 s
measured survival	1.00 s	1.21 s	1.59 s	2.25 s

Table 3.14: m3 survival time equals the analytic free-fall time across a $5\times$ change in the plant’s time constant, to 3–9 %, from a formula containing no controller term. Nothing else moves the number: drive bandwidth 0.8→35 Hz, cadence, oracle velocity, capture gain (more hurts), rear-foot support, CoM lowering, and all 21 gains were swept.

Three mechanisms compose the wall. The stance keyframe starts every episode with the toes 9.1 cm ahead of the centre of mass — a 5.67° backward lean at $t = 0$ (the model’s own comment claims “feet under the CoM”; measurement says otherwise), and correcting it buys exactly the

free-fall time of the residual lean, no more. The single toe-sphere contact gives *zero* centre-of-pressure authority at any ankle stiffness — the ankle at m3 is not weak, it is disconnected. And foot placement is geometrically outrun: with the toe ~ 0.92 m below the CoM, a pitch rate $\dot{\theta}$ sweeps the contact point forward at $L\dot{\theta}$, measured to beat the controller’s corrective foot motion 2:1 until the ± 0.30 m fore-aft workspace saturates in under a second. Escaping m3 therefore needs *either* a corrected keyframe *plus* a foot with real fore-aft extent and an ankle strong enough to use it, or step-on-demand triggering in place of a free-running gait clock. This is a plant conclusion, not a controller one — which is exactly what the identical RL and scripted failures imply.

Two further findings transfer beyond the baseline. The stand keyframe’s control vector is not a static equilibrium — the position servo needs ~ 6 cm more commanded foot extension to hold 15 kg, which RL policies discover silently and hand-built controllers must be told. And *feedforward beats feedback through this drive*: a constant extension offset stands for 20 s where adding ride-height feedback drops it to 7.8 s, because a continuous loop closed through a 0.8 Hz pole against a plant diverging in 0.31 s destabilises. Finally, a methodological rhyme: the baseline’s automatic gain tuner, given the objective $t_{\text{alive}} + 3 \text{ dist}$, immediately exploited it exactly as a policy would — driving 1.6 m/s against a 0.3 m/s command to farm the distance term on the pitch-railed plant. **Reward exploitation is a property of optimisation against an objective, not of neural networks.**

3.8.14 Postscript: the rebuilt robot, and a cap that was never there

On 2026-08-10 the hardware caught up with the analysis, and the training plant was re-issued to match — four changes, three of them the physical robot converging on the model’s recommendations and one a modelling gap that had been open the entire project.

1. **The actuators’ speed was never capped in simulation.** Peak torque had always been limited (at the measured 61.2 / 144.5 N·m), but joint *velocity* was unconstrained — a position servo could crank a joint arbitrarily fast for free. The record shows what that bought: the celebrated k350 pitch-balance winner (Section 3.8.5) “balanced” partly by pattering one foot at 11 Hz with peak thigh speed 23 rad/s, *past the motor’s 22 rad/s no-load ceiling*. The cap is now per joint at the measured no-load output speeds (10.30 rad/s hip roll, 22.01 rad/s cam/thigh) — per joint because the two actuator families differ by $2.1\times$, so any scalar cap is wrong for one of them. One more entry in the pattern of Section 3.4.3: an unconstrained axis is an exploit waiting to be found.
2. **The ankle is now the rigid carbon tube** of Section 3.8.11 — stiff, symmetric in push and pull, no buckling limit — replacing both the soft spring and the PETG rod that modelled as tension-only.
3. **The mass moved the right way:** net -209 g per shin, total 14.72 kg.
4. **The drive was retuned from 0.8 to ≈ 3 Hz**, measured by repeating the swept-sine of Section 3.5.3 after adjusting the boards’ position-loop gains with the vendor tool: left thigh $0.86 \rightarrow 3.35$ Hz, cam $0.85 \rightarrow 2.88$ Hz, with the deliberately untouched abduction axis as the in-run control ($0.74 \rightarrow 0.76$ Hz). The uniform-3 Hz assumption is flagged: abduction is a different actuator and has not been re-measured, so the roll milestones may need re-baselining. The retune softens, but does not retire, the current-mode conclusion of Section 3.5.3 — 3 Hz is still below the 4 Hz top of the gait-frequency range.

The bandwidth bracket that justified the retune. Whether drive bandwidth was really what stood between the robot and walking was settled by a three-arm experiment before the

ladder was relaunched: identical cold m2 runs at 0.8, 3 and 12 Hz drive bandwidth, everything else fixed, measured 25 ms delay and workspace kill retained. At 0.8 Hz — the robot as it was — the policy *shuffles and never walks* (duty factor 1.00, then falls). At 3 Hz it **walks**: 20.75 m at 0.69 m/s with genuine per-foot flight. At 12 Hz — effectively the unlimited drive the whole 2.55 m/s sprint record was trained in — it is *worse* than 3 Hz, folding one leg into the workspace kill. So the measured retune sits, fortuitously, near the sweet spot; the old unlimited-bandwidth plant was not just optimistic but qualitatively misleading; and the check cannot be done by replaying an 0.8 Hz-trained policy at higher bandwidth (tried: it moves faster and falls) — the policy has to be *trained* there.

On this rebuilt plant the scripted controller of Section 3.8.13, re-tuned against a full mission schedule (stand → walk forward → stand → walk backward → stand), completes the mission in one 35 s episode on 3 of 3 seeds with a drive-delivery ratio of 0.98 (against 0.31 on the 0.8 Hz drive) — a small deliverable, but the first controller of any kind to demonstrate command-following locomotion on the plant the robot actually is. Tuning against the full mission mattered: a constant-velocity tune produced a walker that could not stand, the scripted twin of the RL command-curriculum lesson. **The m3 pitch wall survives the retune** — still ≈ 1.0 s, still free fall, for both controller families — confirming it structural rather than bandwidth-limited.

A second, more ambitious scripted controller (a capture-step state machine) was built on the rebuilt plant and *failed*, measuring on the way down four plant facts that any future controller — learned or not — inherits: at 96 % of reach, commanded leg *extension* does not exist (a -65 mm vertical command moves the toe $+85$ mm *forward* and 8 mm down through the IK, so weight support must be joint-space, not task-space); capture steps land on air (the swung foot hovers 1–5 cm high through an entire fall, from rear-workspace IK error plus pitch lift — a ground-seeking touchdown helps but does not suffice); the 4-bar loop-closure constraint itself yields ~ 15 cm of ride height under single-leg load with every actuated joint on target, visible only to a forward-kinematics height loop that includes the passive joints; and the plant’s soft modes, measured by impulse response, put a weakly damped ~ 1 Hz vertical body bounce right where discrete stepping at 2.3 Hz pumps it — the architectural reason the shuffling gait beats “smarter” stepping on this machine.

3.9 Negative results

Several substantial pieces of work in this project produced no usable artefact, and are recorded here deliberately. Each cost real time, each closes a door that would otherwise look open, and in most cases the reason for the failure is more informative than a success would have been.

What was tried	Outcome	Why it is worth knowing
Direct joint-torque RL (Section 3.8.1)	Never converged; abandoned on a bandwidth argument that was later measured wrong	The action space was not the problem. The rejection premise was, and only instrumentation could tell.
Per-step position policy for walking (Section 3.8.2)	Skated, at 32 M steps, behind a healthy reward curve	Reward curves cannot certify a gait. Instrumented gait probes can.
Emulated actuated ankle (fixed torque reflex)	Strong version destabilised the gait; gentle version gave a small non-solving lift	A phase-blind torque fights the gait it stabilises. But the small positive identified the right lever.
Decaying external pitch assist, in four variants	Crutch dependence every time; the unaided policy was worse than no assist at all	Any aid that holds the robot up will be leaned on, and the in-training curve will hide it.
Lowering the simulated centre of mass by 6–12 cm	Slightly worse	Rules out the cheapest candidate plant fix.
Hard velocity, acceleration and cadence limits	Collapsed the warm-started policy	They were fighting a broken internal default (Section 3.8.6), not the gait.
Centre-of-mass identification from the excitation runs (Section 3.5.5)	Rejected: explains ≤ 11 –19 % of the gravity residual, 10 of 18 directions in the null space	The CAD centres of mass stand. Knowing the experiment cannot resolve them prevents fitting the same null space again.
Per-body mass, load-dependent stiction, base tilt, passive-pivot friction	All rejected as explanations of the same residual	The residual is conservative and gravity-correlated; no friction model can produce it.
Pure oscillator generator without a residual channel	Stability without locomotion: 15 m of 72, worst cadence in the set	Refutes the residual channel as the cause of the high-frequency footfall.

Table 3.15: Approaches that did not work, and what each one rules out.

3.10 Deployment

The policies of Section 3.8 ultimately have to run on the Raspberry Pi described in Section 3.2.3, against real drives, with no simulator to reset. Four questions decide whether that is viable: can a non-realtime Linux close a control loop fast enough (Section 3.10.1); can the bus carry the traffic, and what happens when it cannot (Section 3.10.2); what does the motor controller actually accept and report (Section 3.10.3); and — the one this project learned the hard way — when something breaks on hardware, is the failure *investigable* (Section 3.10.5)? The first three were settled by measurement. The fourth was settled by an incident.

3.10.1 Linux without a real-time kernel

The Pi runs a stock, non-realtime kernel. The loop-timing measurement of Section 3.5.6 established that this is sufficient: 200 Hz holds at half budget with millisecond-free jitter, and the drives broadcast at exactly 200 Hz anyway, so there is nothing for a faster loop to read.

One Python-specific detail is load-bearing enough to record. CPython hands the interpreter lock between threads at most every `switchinterval` seconds — 5 ms by default, which is *exactly*

one control period, so the web-interface thread could legally hold the interpreter across an entire control tick. Dropping the interval to 0.5 ms (`sys.setswitchinterval(0.0005)`) took the daemon's late ticks from roughly 10 % to a few percent. The steady state, measured with the control daemon, the 200 Hz IMU thread, the web interface and the flight recorder of Section 3.10.5 all running, is **188 Hz against a 200 Hz target, 5.30 ms mean period, 3 % late ticks**. The missing 6 % is operating-system scheduling, and no lever short of a realtime kernel recovers it.

Where the time actually goes was settled by an A/B under `systemd` overrides, restarting the daemon between arms on the otherwise idle robot:

Configuration	Loop rate	Marginal cost
full stack (recorder + IMU + web UI)	186.7 Hz	—
recorder disabled	186.3 Hz	recorder: −0.01 ms/tick, i.e. free
recorder and sensor thread disabled	194.8 Hz	IMU thread: 0.23 ms/tick

Table 3.16: Component cost of the deployed stack, measured on the robot. The always-on flight recorder costs nothing measurable at 200 Hz; the 200 Hz IMU thread costs a twentieth of the budget.

A methodological trap, again. The first loop-rate figures taken during this bring-up read 60 Hz with 34 % late ticks, and produced a confident (and committed) diagnosis of an expensive failure path in the CAN layer. The diagnosis was wrong: the A/B above shows the suspected path costs nothing. The 60 Hz was the *measurement disturbing the measured* — `scp`, remote compilation, `journalctl` and HTTP polling all running on the Pi's four cores while it was being sampled. This is the same failure mode as the IMU noise figures taken on a swaying rig (Section 3.2.4): the number was real, the attribution was not. The rule that follows: never benchmark the robot over the same SSH session that is deploying to it, and keep the retraction in the history rather than rewriting it.

Process supervision. The daemon runs under `systemd`: started at boot, restarted on crash (bounded to five restarts per minute, so a crash loop stays visibly down instead of power-cycling the motors), stdout in the journal, and stopped by `SIGINT` so the shutdown path runs — which streams zero current to every motor and writes the recorder's clean-stop record, whose *absence* is how a power cut is later inferred (Section 3.10.5). Before this, the only record of the daemon's death was whatever terminal happened to still be open.

3.10.2 CAN at runtime: the bus with nothing on it

The protocol itself is described in Section 3.5.7; bandwidth is a non-issue (26 % of one of two 1 Mbit buses). What the protocol description does not cover is the failure mode that running under `systemd` made routine: **the daemon now starts before the motor power is on**. With nothing on the bus to acknowledge a frame, `socketcan`'s ten-frame transmit queue fills within one tick and every subsequent `send()` raises `ENOBUFS`. On its first boot under `systemd`, that exception — unhandled, in a background thread — killed the control daemon outright during its own preflight. The bug predates the service; hand-started sessions had simply always had the motors powered first.

The fix is that a send failure is an *event*, *not a death*: sends never raise past the CAN layer, failures are counted per channel, and a channel that fails ten consecutive sends is retried at 5 Hz instead of 200 — one successful send restores full rate. Two honest measurements qualify this. First, a failing send costs nothing: with the backoff disabled and three sends per tick failing,

the loop runs at 188.9 Hz against 188.5 with it enabled. The backoff is kept for hygiene (600 pointless system calls per second) and for forensics — “we did not attempt the frame” and “we attempted it and it failed” are different facts, and the recorder logs both. Second, safety does not rest on the backoff: motors on a dead bus report nothing, so the motion interlocks refuse to command them, and a bus that starts refusing frames *while* the robot is being commanded trips the safety stop within half a second.

3.10.3 The motor controller as deployed

Three properties of the CubeMars drives dominate the deployment design, all three measured rather than documented (Sections 3.5.3 and 3.5.7).

First, the only writable surface is the servo-mode command pair (`SET_POS`, `SET_CURRENT`); the internal cascaded gains are board configuration, readable off the vendor tool but not over CAN, and the position loop they implement is proportional-only with a 0.8 Hz corner. Second, the drives report no bus voltage, so pack sag under load is unobservable from the drive side. Third — and the property with the sharpest consequences — **the multi-turn position origin is not stable**: it re-randomises on every power cycle, and the incident below is consistent with it moving *between* power cycles as well. Every absolute position command is therefore only as good as a calibration captured earlier in the same session, and nothing in the drive signals when that calibration has silently become wrong.

3.10.4 The safety envelope as an instrument

A browser-based control interface runs on the robot and is the only path through which the hardware is commanded by hand. It is treated here as methodology rather than as tooling, because the safety envelope it enforces is also an experimental instrument.

- **Measured joint limits, not CAD limits.** Each joint is backdriven by hand through its full travel with the motors limp and the excursion recorded, giving a per-joint range that reflects the assembled machine including cabling and interference. The 4-bar’s assemblable band is a joint *pair* constraint, so the recorded reachable region is a box in the foot’s fore-aft and lift coordinates rather than a set of independent joint ranges.
- **The recorded workspace is reused in simulation.** The same measured box is applied as a termination criterion during training: an episode ends the moment a foot leaves the region the real robot is known to reach. This began as a safety measure and became a sim-to-sim gate, because it makes an entire class of simulator-only exploit impossible rather than merely expensive. It also has to be applied carefully, since the criterion is expressed in the base frame and therefore cannot by itself distinguish a collapsed ankle from a folded linkage (Section 3.4.3).
- **Playback is capped by kinematics, not by preference.** After a part failed during a high-cadence trajectory playback, the frequency ceiling was reduced from 5 Hz to 2.5 Hz, and the number was derived rather than guessed. Differentiating the stored waveform gives the peak joint speed per unit cadence as a property of the trajectory *shape*: the cam joint demands 359 deg/s per Hz and reaches the actuator’s no-load speed at 3.5 Hz. Past that the command is not merely hard to track, it is kinematically impossible, and the drive simply saturates current chasing a target it cannot reach. At the 5 Hz previously allowed, the cam was being commanded at 143 % of no-load speed. The 2.5 Hz ceiling leaves it at 71 %. Raising it again requires a hardware argument, and because the limit depends on the shape, a gentler trajectory can safely go faster.
- **Trip conditions.** Tracking error, joint range and temperature are monitored continuously and latch the drives to zero current. The tracking-error trip in particular had to be decoupled

from the commanded trajectory once Section 3.5.3 showed that a large tracking error at speed is the *expected* behaviour of this drive rather than a fault.

3.10.5 Hardware validation: the flight recorder and the pre-move guard

On 2026-08-10 a joint destroyed itself on the bench. A zero pose had been captured, the drives' raw origin no longer matched it by the time an absolute move was commanded, and the drive — faithfully executing a position command in the wrong frame — wound `left.cam` through roughly 1.9 output turns on a joint with $\pm 86^\circ$ of travel. The investigation was *impossible*: the only telemetry in the system was a 4096-sample RAM ring at 20 Hz, of which the HTTP interface could serve the last 25.6 seconds, and nothing was on disk. By the time anyone looked, the window had rolled past the event. The remainder of this section describes what was built in response, because “make the failure investigable” turned out to be a design problem with measurable requirements, not a logging afterthought.

A three-tier flight recorder. Streaming the full state to disk continuously is not an option: six motors at 200 Hz is 35 kB/s, 3 GB/day, which fills and then destroys an SD card. The recorder therefore splits by rate:

Tier	Content	Mechanism
A	every field, continuously	20 Hz, fixed-width binary segments, rotated under a global disk budget
B	the black box proper: the full 200 Hz state	a 40 s RAM ring, dumped to its own file on any trigger, <i>pre-trigger history included</i> , plus a 3 s post-trigger tail
C	one JSON line per event	append-only, <code>fsync</code> 'd on write, never deleted

Table 3.17: The flight recorder. Tier B triggers include every safety trip, e-stop, mode transition, calibration or configuration write, warn-level tracking error (below the trip threshold, so the *approach* is captured, not only the fall), and an encoder-origin jump. Each dump embeds a content-hashed snapshot of the calibration and drive gains live at that instant, so a recording is self-describing without external context.

Two constraints shaped the implementation. The control loop must not pay for its own instrumentation: the 200 Hz thread only appends a tuple to a bounded queue ($\approx 2 \mu\text{s}$, and Table 3.16 confirms the end-to-end cost is unmeasurable), a separate writer thread owns all file I/O, and a full queue *drops and counts* — the running drop count is stamped into the next accepted record, because a black box that cannot prove where its own gaps are is worse than none. And the robot must never depend on the recorder: if the writer thread dies, the control loop continues and the failure is surfaced in the interface.

Time, without a trustworthy clock. The Pi has no RTC and its access point has no internet, so the wall clock after boot is whatever `fake-hwclock` restored — plausible and wrong. Every record therefore carries the monotonic clock as its primary time base, the wall time plus an explicit *trusted* flag (set only once NTP has actually synchronised), a per-boot kernel identifier and a per-run session identifier, so records from different sessions can never be silently interleaved. When NTP does land, the resulting step is itself logged (+1333.5 s on the first instrumented boot), so earlier stamps can be corrected offline. Power-off is handled by inversion: a process cannot log its own death, so a heartbeat is `fsync`'d every 8 s, and the last heartbeat before a gap

is the power-off time, to heartbeat resolution — an inference the offline reader states explicitly rather than implying a clean shutdown.

The pre-move guard. The incident’s mechanism — a full-authority slew against a stale calibration — is closed by four mechanisms, none of which adds a step to the normal workflow. That last clause is a requirement, not a nicety: an early draft forbade homing as the first move after calibration, and that was exactly backwards, because after a zero capture the legs are limp and sagging and homing is precisely what catches them.

1. **Hold before moving.** The first command after enabling is “hold the raw position you are reporting right now”, for 0.35 s. It never consults the calibration, so it is safe even against a completely wrong zero, and it arrests the sag immediately. Only then does the slew to an absolute target begin.
2. **Raw-at-rest fingerprint.** The raw encoder pose of all six motors is recorded at every successful zero capture. Before leaving the limp state, the live raw pose is compared against it: decisive after a restart (the origin re-randomises, so a calibration restored from disk is refused until re-zeroed), while within a live session a 200 Hz continuity watchdog is used instead — an origin *rewrite* moves the reported position without the drive reporting speed, which is exactly how it is distinguished from the joint being moved by hand. A range-sanity check backstops both: a calibration implying 678° on an $\pm 88^\circ$ axis is refused outright.
3. **A homing-specific tracking threshold** (8° rather than the 25° playback value): a slow guided slew should track well, so a stale zero appears as tracking error within the first degrees of travel.
4. **A per-joint travel budget** as the final backstop: a guided move that sweeps more than $1.3\times$ the joint’s own range without arriving is stopped. On 2026-08-10 this alone would have cut the excursion from 1.9 output turns to under 0.6.

Acceptance. The incident shape is reproduced in a hardware-free test: the mock bus grew an `origin_shift` fault injection that moves the reported *and* commanded frame while the shaft stays put and reports zero speed — which is what a board renumbering its origin actually looks like, and what makes it invisible to any check that trusts reported velocity. The test captures a zero, shifts the origin, commands a home, and asserts that the guard refuses with both raw poses on record, that the triggered dump contains at least ten seconds of pre-trigger 200 Hz data, and that the offline reader’s `--postmortem` mode states what happened — boot and power-loss times, the raw-pose comparison, commanded versus reported position through the event, the configuration live at that instant, and whether the origin ever moved while the robot was limp and untouched — without a human opening a raw file.

What it found in its first hour. The recorder justified itself before the day ended. Its first boot on the robot captured its own deployment failure — the `ENOBUFFS` crash of Section 3.10.2, complete with traceback, which had previously been invisible — and its steady-state telemetry established that the left-leg bus was electrically dead (zero frames transmitted, all three motors silent), turning “the robot feels wrong” into a specific, timestamped fact. It also ended a longer-standing blindness: nothing in the stack had ever measured the control loop’s actual rate, so a loop running at a third of its nominal frequency had been indistinguishable from a healthy one. The tick counter the recorder stamps into every sample is what made every figure in Table 3.16 measurable at all.

3.11 Hardware experiments

Everything in this section ran on the physical robot, through the web instrument of Section 3.10; the heavy measurement results are reported in Section 3.5 and are only situated here.

In-air gait demonstration and teach-and-replay. The first hardware milestone was a hand-authored straight-walking gait with the body clamped in the air (no balance): a pure-NumPy gait generator streaming servo-mode position commands over CAN. Its safer successor is teach-and-replay: the motors are held limp by streaming zero current, a leg is backdriven by hand through the desired cycle, takes are recorded per side, and a pure-NumPy pipeline resamples, phase-aligns, detects the left/right mirror, FFT-low-passes the loop closed, and replays it — either as drive-side position commands or through a software PID onto current commands with a hard ampere clamp as the torque limit. Replaying on the robot exposed the compliance problem that later measurements explained: at low proportional gain the leg drew only ~ 2 A against a 20 A allowance and moved with stick-slip judder; the practical fix (raise k_p , add k_i against the gravity lag, derivative on target-minus-actual velocity) was an early, unquantified encounter with the drive-bandwidth ceiling of Section 3.5.3.

Workspace calibration. The safe joint-space workspace was recorded by backdriving each leg through its physically assemblable region and eroding the recorded polygon; it does triple duty as the training-time constraint, the runtime watchdog, and the offline deploy gate. Calibration on hardware corrected the model in one substantial way: **the cam is a full crank**. The CAD-guessed joint range of $\pm 86^\circ$ is wrong — recorded sweeps span $\sim 245^\circ$ of cam rotation, the 4-bar assembling at every angle — and the calibrated zero pose sits $\sim 1^\circ$ outside the eroded safe band (so a naive hold-at-zero is refused by the safety net while a guided homing move is allowed through the physical-feasibility check instead; Section 3.10.5).

The identification campaign. Quasi-static and swept-sine excitations, captured at 200 Hz through the measurement mode of the web instrument, produced the plant corrections of Section 3.5: the known-mass K_t measurement, the 0.8 Hz drive Bode, the gravity-model residual and its rejected explanations. The campaign also demonstrated the instrument itself: runs are self-describing (calibration, drive gains and weighed masses embedded in each capture), and the excitation planner solves amplitude against travel, no-load speed and predicted tracking error jointly after an early sweep e-stopped the run six seconds in by asking the drive for more than its position loop can execute.

Sensor bench. The IMU characterisation of Section 3.2.4 (noise, latency, filter behaviour, and the motion-gating methodology that its first wrong numbers forced) was performed in place on the robot, with the provenance-labelling scheme described there.

The incident, and what it built. On 2026-08-10 a stale calibration met a full-authority absolute move and destroyed a joint (Section 3.10.5). The investigation being impossible was itself the finding: it produced the flight recorder, the pre-move guard, and the systemd supervision of Section 3.10, all validated by reproducing the incident against the mock CAN bus. The recorder's first hour on hardware then found a daemon-killing crash at boot (unpowered CAN bus) and established that the left-leg bus was electrically dead — the current state of the machine as this is written: right leg fully functional under the 200 Hz stack at 188 Hz measured, left leg awaiting repair of the failed joint.

3.11.1 Static end-effector force map

The cam actuator reaches the knee through the pushrod 4-bar, so the transmission from motor torque to toe force is a strong function of posture. Whether the strong direction is the one running needs had never been checked.

The constrained toe Jacobian differentiates the loop closure out, so it carries the linkage's transmission ratio, and statics against the actuator torque box gives the feasible force set at every reachable toe position. Self-tests check loop closure against the stance keyframe, the analytic Jacobian against central differences, and a fast analytic ankle column against the full Jacobian.

Three findings, all statics-only and therefore bounding capability rather than predicting behaviour:

- **The leg stands at 96 % of its reach**, with about 3mm of further extension available anywhere near the stance position. Everything else follows from sitting that close to the leg-extension singularity.
- **Force there is 19:1 anisotropic**: 16.4 body weights straight down, 0.85 body weights horizontally. This is the same near-singular corner the speed-ceiling analysis found from the power side, seen from the force side. It is a property of the fully-extended pose rather than of the whole workspace: over the running stance region the median tilted capacity recovers to 5.95 BW, because compressed postures rotate their strong axis.
- **The motors are not the binding constraint; the ankle is, by a factor of eight**. Over the stance region the actuators give a 4.30 BW median vertical capacity, and 77 % of the region clears the 3.5 BW running requirement. The passive ankle gives a 0.42 BW median, and **0.0 %** of the region clears it.

The last point is a third independent route to the conclusion of Section 3.8.5 and Section 3.6, reached from statics with no learning and no dynamics: the linkage and motor sizing are adequate to generous, and the foot spring is the design defect.

4 Results

4.1 Hardware

The delivered hardware platform: a 15.14 kg biped as weighed — 14.72 kg after the carbon-ankle rebuild (Section 3.8.14) — with six quasi-direct-drive actuators (Table 3.1), a 13S2P / 480 Wh / 250 A pack (Section 3.2.2), a Raspberry Pi 3B closing a 200 Hz control loop at 3 % late ticks on stock Linux with half its budget spare (Tables 3.6 and 3.16), dual 1 Mbit CAN at 26 % utilisation, an IMU characterised to provenance-labelled noise figures (Table 3.2), and a measured plant model whose every correction is tabulated in Table 3.4. The capability analysis on the corrected plant places the machine at ≈ 2.7 m/s credible burst and ≈ 1.3 m/s thermally sustained (Section 3.6.6). The deployment stack — calibration, safety guards, flight recorder — runs on the robot under systemd and has survived (and diagnosed) its first real incident (Section 3.10).

4.2 RL experiments

Table 3.11 remains the headline: the milestone ladder culminating in a 6-DOF-free plant that runs 60 s without falling at 2.6 m/s, and the cadence-fixed m7 at 2.71 m/s — ≈ 70 % of the (pre-correction) credible ceiling. Around it, the consolidated findings:

- **Action space:** direct torque never converged and was undeployable as assumed (Section 3.8.1; premises later partially overturned by measurement); per-step position targets converged to standing but skated (Section 3.8.2); the per-step-reparameterised Fourier CPG with reflexes and residuals is the design that works (Section 3.8.3).
- **Generator A/B:** oscillator-CPG vs Fourier is a measured tie with complementary strengths; the oscillator produced the single best controller (the only one that executes the sprint’s stop phase) but walls at the roll DOF where Fourier passes (Section 3.8.10).
- **The pitch wall is the plant’s:** identical ~ 1 s failures for RL and for a 21-gain classical controller, equal to the analytic free-topple time across a $5\times$ gravity sweep (Section 3.8.13); broken in simulation only by a $12\times$ ankle stiffening (Section 3.8.5); and it survives the hardware drive retune to 3 Hz (Section 3.8.14) — structural, not bandwidth.
- **On the rebuilt plant** (carbon ankle, 3 Hz drive, velocity caps), the scripted controller demonstrates the first full command-following mission — stand, walk forward, walk backward, stand again — 35 s, 3/3 seeds, drive delivery 0.98 (Section 3.8.14).
- **On the measured plant,** the sim-to-real lineage completes the descent to the real 0.8 Hz drive with a $3\times$ score gain over its parent, and its diagnosis history (steering asymmetry, DR ablation, command-range fit) is itself a result (Section 3.8.12).
- **The ankle question closed as a build decision:** statics eliminate the as-built soft spring, learning selects the compression-capable strut, and buckling arithmetic rejects the fabricated part in favour of carbon (Section 3.8.11).

5 Discussion

5.1 Successes

The pipeline exists end to end and each stage has been validated at least once: CAD to a loop-closed MJCF that matches the measured workspace at 100 % of poses; a capability analysis that independently reproduces both the RL-discovered high-cadence optimum and the RL-discovered ankle-stiffness threshold; an action-space design that turned a non-converging problem into a reliably trainable one, twice confirmed by an independent generator A/B; a curriculum mechanism (base-DOF release with rigid rails and constant observation/action shape) that carries warm starts across plants; a measurement campaign that corrected the plant with instruments built on the robot itself; and a deployment stack whose flight recorder proved its value within an hour of installation. The standing policy on the fully free plant — 60 seconds, zero falls, 2.6 m/s — is a real result on a hard plant, and the drivable lineage’s survival of the descent to the measured 0.8 Hz drive is the strongest sim-to-real evidence in the record.

5.2 Failures

Direct torque control failed twice over (convergence and assumed deployability) and its abandonment was itself partially wrong — the measured premises now point back toward current-mode control with the PD on the Pi. Fully autonomous untethered running was not reached: the sprint policies observe privileged state, the drivable lineage is not yet robust, no policy has run on the robot, and the record ambition is dead on this chassis by measurement (≈ 1.3 m/s sustained against a 4 m/s record pace). The roll milestone remains the RL ladder’s weak link; the 6 Hz footfall chatter survives every post-hoc lever and awaits a from-scratch retrain with cadence pressure from step zero; and the left leg of the physical robot was lost to a calibration failure before any policy reached it — the clearest possible demonstration that deployment infrastructure is not overhead.

5.3 Lessons learned

1. **Optimisers exploit objectives; neural networks are not the culprit.** The reward was gamed by PPO (skating, floor penetration, shuttling, penalty-suicide), and the scripted baseline’s coordinate-descent tuner gamed its objective identically. Every objective needs adversarial review before its optimiser gets it.
2. **Every assumed plant parameter measured later moved pessimistically** — mass +18%, torque −15%, drive bandwidth $\times 16$, friction, the ankle. Measure before training, not after; and when a measurement lands, apply it as *the* plant, not as an optional variant (the wrong default will otherwise be trained on).
3. **Simulation results need the same scepticism as rewards:** soft joint limits, constraints inactive at settle, frame-relative safety checks, stale keyframes, a solver-version split, and an actuator-velocity cap that was simply never present (letting a prized policy “balance” at 23 rad/s on a 22 rad/s motor) each produced a clean, converged, wrong result before being caught by self-tests, smoke gates, or measurement.
4. **Evaluation hygiene is a first-class deliverable:** aid-off evaluation (three deceptions by training curves with a crutch), fixed-task evaluation across curricula, ≥ 20 seeded episodes, paired seeds with survivor counts under bimodality, peak-not-final under one-way curricula, and `curriculum.json` as ground truth for restore. Four separate evaluation bugs each nearly discarded or crowned the wrong policy.

5. **Curricula: gate the advance, not just the opening; never gate a rescue ramp on the competence it is meant to rescue; and diff configs against the last working run before theorising.**
6. **A parameter range widened “to be non-binding” can become the operating point** — the neutral network output decodes to the middle of the range (the 25 Hz gait-clock bug), and a comment asserting an unexercised assumption stays wrong for months.
7. **Hardware integration dominates, and instrumentation pays for itself:** the measurement instrument found the drive ceiling; the flight recorder found a daemon-killing crash and a dead bus in its first hour; and the measurement-disturbs- the-measured trap was walked into twice (IMU on a swaying rig, loop rate under SSH load) — the second time producing a committed wrong conclusion that only an A/B retracted.
8. **Progressive validation works.** The base-DOF ladder localised failures to single degrees of freedom; the toy problem debugged the deploy chain at one DOF; L0-style telemetry cross-checks and the workspace-as-safety-net made hardware sessions survivable — with the one incident occurring precisely in the gap the guard now closes.

6 Conclusion and Future Work

6.1 Summary of Findings

This thesis set out to apply reinforcement learning to a custom high-speed biped and delivered, in the end, three things: a working method (the modelling, training and curriculum stack that produces a 2.6 m/s never-falling runner on the free simulated plant), a measured machine (every load-bearing parameter of DASH-01 replaced by a measurement, with stated uncertainty and stated un-resolvables), and an honest account of the distance between the two. The distance is not rhetorical: the measured robot is heavier, weaker and — decisively — an order of magnitude slower in actuation than the trained-against model, and its corrected ceiling (≈ 2.7 m/s burst, ≈ 1.3 m/s sustained) reclassifies the platform from record contender to demonstrator. The learning method itself is vindicated where it can be tested: it matches or is matched by classical control exactly at the plant’s own limits, which is what a healthy learning stack should do.

6.2 Challenges and Limitations

The limits of the present work are recorded throughout and summarised once: no policy has yet run on the hardware (export and inference on the Pi remain the critical path); the sprint results rest on privileged observations; the link inertia tensors and the ~ 0.9 N·m gravity residual remain unresolved by identification, with the kinematic model the remaining suspect; the actuator model lacks a coupled torque–speed envelope, optimistic exactly at high speed; the capability analysis’s corrected figures are a sensitivity propagation awaiting a full re-sweep; and the physical robot currently stands with one functional leg.

6.3 Suggestions for Future Research

For *this* robot, the priorities are ordered by the record: ONNX export and a Pi inference loop (everything else waits on it); current-mode control with the position PD closed on the Pi at 200 Hz, replacing the 0.8 Hz drive loop that every measurement identifies as the binding constraint; the carbon ankle strut; and the from-scratch cadence retrain.

For a *successor* robot, the measured record ranks the design levers:

1. **Bus voltage before torque:** 87 % of feasible gaits are limited at the cam’s peak-power corner, and speed scales linearly with no-load speed and not at all with peak torque.
2. **Distal mass:** at -0.93 (m/s)/kg, nine times the leverage of torso mass; a carbon shin and a proximally mounted spring on a tendon (a true Achilles) recover ≈ 0.5 m/s for free.
3. **Re-proportion or replace the 4-bar:** only 0.20–0.35 m of a 1.06 m geometric span is usable, and the standing pose sits at 96 % of reach where the toe-force ellipse is 19:1 anisotropic — strong vertically, weak in exactly the propulsive direction. Target a flat transmission ratio over ~ 0.5 m with stance at ~ 85 % of reach.
4. **Size the ankle for 3.5 body weights** — the present passive ankle delivers a median 0.42 BW over the running-stance region, $8\times$ short, in positive mechanical feedback.
5. **Cool the cam motor:** sustained speed is what a 100 m record is made of, and the gap between 167 N·m demanded and 55 N·m continuous is thermal.
6. **Torque-mode control as an architectural requirement from day one** — the v1 lesson, twice paid for.

Finally, the validated toe-Jacobian force map and the capability sweep make a *design search in*

simulation — over bus voltage, gear ratio, distal mass, linkage proportions and ankle stiffness — the natural next use of the training cluster: searching over robots, rather than over controllers for a robot already known to be the constraint.

7 Appendix

7.1 logbook

- 2025-02-20** Creation of this report, rough layout of the idea in chapter
- 2025-02-25** Longitudinal kinematic model, research about markov modelling and cycle
- 2025-02-26** dynamic simulation creation, implemented a basic simulation in pybullet with keyboard navigation and parametric surface + wheel rolling down
- 2025-02-27** refining efficiency to take into account whole cycle, reading paper
- 2025-02-28** Writing Idea about improving efficiency through identified parameters
- 2025-03-04** Adding graph about car occupancy and real world driving needs + reading paper
- 2025-03-06** Brainstorming kinematic, reading PhD thesis on narrow track
- 2025-03-09** URDF is a PAIN to implement to due lack of tool, trying to define a kinematic model in python
- 2025-03-11** reworking part of chapter 2
- 2025-03-17** reading paper about cars and bike and limiting factor to adoption
- 2025-03-18** keep reading paper, also about narrow track vehicle
- 2025-03-20** trying to structure what I learned from the papers
- 2025-03-21** defining the know unknown about car adoption factor (human side)
- 2025-03-24** Clarifying the project goal
- 2025-03-27** draw figure, added citation
- 2025-03-28** Manual definition is an epic failure... Two way (createMultiBody -; Works but hard to read), Create body and add constraint don't work (rotation constraint crash the simulation.)
- 2025-04-09** Realized that createMultibody is more stable but has a "ghost" friction problem, Might as well use URDF then.
- 2025-04-10** learning URDF and how the origin is defined,
- 2025-05-25** redefined most variant as URDF and how to simulate them. Need spend time to implement basic controller like PID to show steady state stability and control margin
- 2025-05-28** Meeting, dry run presentation and shown current status.
- 2025-05-31** Report restructuring, writing
- 2025-06-02** Chapter 4, 5, 6 writing
- 2025-06-04** Getting simulation result, finishing first draft of report.
- 2025-06-08** Proof reading

7.2 URDF from CAD:step by step

recall limitation of URDF regarding parallel robot, tricks with closed loop and equality constraint, other format that are better suited but did not manage to export. add screenshot and show how to do it for the full robot.

References

- [1] Guillaume Bellegarda and Auke Ijspeert. CPG-RL: Learning central pattern generators for quadruped locomotion. *IEEE Robotics and Automation Letters*, 2022. arXiv:2211.00458.
- [2] Devin Crowley, Jeremy Dao, Helei Duan, Kevin Green, Jonathan Hurst, and Alan Fern. Optimizing bipedal locomotion for the 100m dash with comparison to human running. *arXiv preprint*, 2025. arXiv:2508.03070.
- [3] Atil Iscen, Ken Caluwaerts, Jie Tan, Tingnan Zhang, Erwin Coumans, Vikas Sindhwani, and Vincent Vanhoucke. Policies modulating trajectory generators. *Conference on Robot Learning (CoRL)*, 2018. arXiv:1910.02812.
- [4] Jonah Siekmann, Yesh Godse, Alan Fern, and Jonathan Hurst. Sim-to-real learning of all common bipedal gaits via periodic reward composition. *IEEE International Conference on Robotics and Automation (ICRA)*, 2021. arXiv:2011.01387.